



LATIF JI BOLI

**A SINDHI PROGRAMMING LANGUAGE
INTERPRETER**

Muhammad Muzamil (22SW30)

Ali Yawar (22SW43)

Ahsan Abbas (22SW12)

A thesis submitted in fulfillment of the
requirements for the award of the degree of

Bachelor of Engineering

in

Software Engineering

Department of Software Engineering

Faculty of Software Engineering

Quaid-e-Awam University of Engineering, Science & Technology

Nawabshah

2026

DECLARATION

We hereby declare that the thesis entitled “**Latif Ji Boli** – A Sindhi Programming Language Interpreter” is our original work, completed as part of the requirements for the degree of Bachelor of Engineering in Software Engineering. The work presented in this thesis, including the design, development, implementation, and testing of the proposed Sindhi Programming Language Interpreter, has been carried out by us under the guidance of our supervisor.

Signature: _____

Name: Muhammad Muzamil (22SWE30)

Signature: _____

Name: Ahsan Abbas (22SW12)

Signature: _____

Name: Ali Yawar (22SW43)

Date: _____

DEDICATION

I want to dedicate this thesis to my parents. They have always been there for me. Their prayers and love mean a lot to me. They made sacrifices so I can do what I want to do. This has always motivated me to keep going and reach my goals. I also want to dedicate this work to my teachers and mentors. They helped me a lot and gave me good guidance and support when I needed it. Finally, I dedicate this project to all Sindhi-speaking learners who want to learn about technology and programming. I hope this project will help them in some way.

CERTIFICATE

QUAID-E-AWAM UNIVERSITY OF ENGINEERING, SCIENCE & TECHNOLOGY NAWABSHAH

This thesis, written by **Mr. Muhammad Muzamil, Mr. Ali Yawar, and Mr. Ahsan Abbas** under the direction of their supervisor and approved by all the members of the thesis committee, has been presented to and accepted by the Dean, Faculty of Software Engineering, in fulfillment of the requirements of the degree of Bachelor of Engineering in Software Engineering.

Supervisor: Ali Raza Bhangwar

Internal Examiner

External Examiner

Director, Postgraduate Studies & Research

Dean, Faculty of Engineering

Date: _____

ACKNOWLEDGMENTS

Thanks to “**ALMIGHTY ALLAH**” for granting me the strength, wisdom, and opportunity to accomplish this great achievement. I made every effort to achieve this goal with the continuous support and cooperation of my family members, teachers, and friends.

I sincerely acknowledge that without the guidance and support of my respected teachers, I would not have been able to accomplish this task. I express my deepest gratitude to my supervisor, **Dr. Ali Raza Bhangwar**, for his valuable guidance, suggestions, encouragement, and continuous support throughout the development of this project. His constructive feedback and critical analysis played an important role in the successful completion of this work.

I am especially thankful to my supervisor for his valuable suggestions, critical analysis, and guidance throughout the design, development, and implementation of the Latif Ji Boli Sindhi programming language.

Last but not least, I am thankful to all the faculty members of the Department of Software Engineering, QUEST, for their cooperation, support, and friendly environment throughout my undergraduate studies.

TABLE OF CONTENTS

DECLARATION.....	ii
DEDICATION.....	iii
CERTIFICATE	iv
ACKNOWLEDGMENTS	v
TABLE OF CONTENTS.....	vi
LIST OF TABLES	viii
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS.....	x
ABSTRACT	xi
CHAPTER 1: INTRODUCTION	1
1.1 BACKGROUND	1
1.2 PROBLEM STATEMENT	4
1.3 RESEARCH MOTIVATION	6
1.4 RESEARCH AIM	7
1.5 OBJECTIVES	8
1.6 SCOPE OF THE STUDY	10
1.7 SYSTEM OVERVIEW	12
1.8 FEATURES OF THE PROPOSED SYSTEM.....	14
1.9 EXPECTED OUTCOMES	17
1.10 CHAPTER SUMMARY	18
CHAPTER 2: LITERATURE REVIEW	20
2.1 BACKGROUND	20
2.2 EVOLUTION OF PROGRAMMING LANGUAGES.....	21
2.3 COMPILER AND INTERPRETER TECHNOLOGIES	22
2.4 LEXICAL ANALYSIS IN COMPILER DESIGN	23
2.5 TOKENIZATION AND TOKEN MANAGEMENT	25
2.6 UNICODE SUPPORT IN REGIONAL LANGUAGE PROGRAMMING	26
2.7 REVIEW OF NATIVE LANGUAGE PROGRAMMING SYSTEMS	27
2.8 COMPARATIVE ANALYSIS OF EXISTING SYSTEMS.....	28
2.9 RESEARCH GAP.....	29
2.10 CHAPTER SUMMARY	29
CHAPTER 3: RESEARCH METHODOLOGY	31
3.1 BACKGROUND	31
3.2 RESEARCH METHODOLOGY	32
3.3 RESEARCH DESIGN	33
3.4 REQUIREMENT ANALYSIS	34
3.5 SOFTWARE DEVELOPMENT METHODOLOGY	35
3.6 SYSTEM DESIGN	36
3.7 SYSTEM ARCHITECTURE	38
3.8 IMPLEMENTATION METHODOLOGY	39
3.9 TESTING METHODOLOGY	41
3.10 TOOLS AND TECHNOLOGIES USED	42
3.11 WORKING METHODOLOGY OF THE PROPOSED SYSTEM.....	43
3.12 ADVANTAGES OF THE SELECTED METHODOLOGY	45
3.13 CHAPTER SUMMARY	46

TABLE OF CONTENTS

CHAPTER 4: RESULTS AND DISCUSSION	48
4.1 INTRODUCTION	48
4.2 SYSTEM IMPLEMENTATION RESULTS	49
4.2.1 Lexical Analysis Results.....	50
4.2.2 Variable Declaration and Assignment	51
4.2.3 Arithmetic Expression Evaluation	52
4.2.4 Conditional Statement Execution	53
4.2.5 Loop Execution.....	54
4.2.6 String Handling.....	55
4.2.7 Error Handling and Runtime Validation	56
4.3 DISCUSSION OF RESULTS.....	57
4.4 FUNCTIONAL EVALUATION OF THE SYSTEM.....	59
4.5 ADVANTAGES OF THE DEVELOPED SYSTEM	60
4.6 LIMITATIONS OF THE DEVELOPED SYSTEM	60
4.7 CHAPTER SUMMARY	61
CHAPTER 5: CONCLUSION AND FUTURE WORK	62
5.1 BACKGROUND	62
5.2 CONCLUSION.....	62
5.3 ACHIEVEMENT OF RESEARCH OBJECTIVES	63
5.4 CONTRIBUTIONS OF THE RESEARCH.....	64
5.5 FUTURE WORK.....	65
5.5.1 Support for Advanced Data Types.....	65
5.5.2 Functions and Procedures	66
5.5.3 Parser and Semantic Analysis.....	66
5.5.4 Integrated Development Environment (IDE).....	66
5.5.5 Compiler Optimization	66
5.5.6 Educational Integration.....	67
5.5.7 Multilingual Programming Support.....	67
5.6 FINAL REMARKS	67
REFERENCES	69

LIST OF TABLES

Table 1.1 Major System Components	12
Table 1.2 Supported Programming Features	14
Table 1.3 Data Types and Scope of the Proposed Language	17
Table 1.4 Program Execution Flow	12
Table 2.1 Comparative Analysis of C, Python, and Latif Ji Boli	28
Table 3.1 Research Methodology Stages	32
Table 3.2 Tools and Technologies Used	42
Table 4.1 Functional Evaluation of Latif Ji Boli	59
Table 4.2 Error Handling and Validation Scenarios	57
Table 5.1 Future Enhancement Areas	65

LIST OF FIGURES

Figure 1.1 Research Problem Diagram	5
Figure 1.2 Research Objective Mind Map	9
Figure 1.3 Scope Diagram.....	11
Figure 1.4 Overall System Architecture.....	13
Figure 1.5 Compiler Working Flow	16
Figure 3.1 Activity Diagram	32
Figure 3.2 Use Case Diagram	33
Figure 3.3 Sequence Diagram	36
Figure 3.4 Class Diagram.....	38
Figure 3.5 Component Diagram.....	39
Figure 3.6 Development Diagram.....	42
Figure 3.7 Data Flow Diagram (Level-0).....	43
Figure 3.8 Data Flow Diagram (Level-1).....	44
Figure 4.1 Compiler Pipeline Diagram	48
Figure 4.2 Lexer Internal Flowchart.....	49
Figure 4.3 Variable Declaration and Assignment Output	50
Figure 4.4 Arithmetic Expression Evaluation Output	51
Figure 4.5 Conditional Statement Execution Output.....	52
Figure 4.6 Loop Execution Output.....	53
Figure 4.7 String Handling Output.....	54
Figure 5.1 Future Enhancement Roadmap	63

LIST OF ABBREVIATIONS

API	—	Application Programming Interface.....
ASCII	—	American Standard Code for Information Interchange.....
AST	—	Abstract Syntax Tree
DSR	—	Design Science Research.....
HTTP	—	Hypertext Transfer Protocol.....
IDE	—	Integrated Development Environment.....
REST	—	Representational State Transfer

ABSTRACT

Programming is an important skill in today's digital world, but most programming languages use English-based keywords and syntax. This can make programming difficult for students who are more comfortable with their native language. For Sindhi-speaking learners, the need to understand English programming terminology can become an additional barrier to learning programming concepts [18], [19]. To address this issue, this project presents Latif Ji Boli, a Sindhi programming language interpreter developed in Java. Latif Ji Boli allows users to write and execute programs using Sindhi keywords and Unicode-based Sindhi script. The interpreter performs lexical analysis by converting source code into tokens such as keywords, identifiers, numbers, strings, operators, and symbols. It then processes these tokens and executes the program according to the rules of the language. The language supports basic programming features including variables, numeric and string data types, arithmetic and logical operations, conditional statements, while and do-while loops, comparison operators, and output statements. It also provides syntax and runtime error messages with line and column information to help users find and correct errors. The system uses a modular architecture with separate components for lexical analysis, token management, and interpretation, making it easier to maintain and extend. Java was selected because of its portability and suitability for developing a structured programming language system [12], [13]. Overall, Latif Ji Boli demonstrates that Sindhi can be used effectively for programming while providing a more familiar learning environment for Sindhi-speaking beginners. The project also provides a foundation for future improvements, such as additional language features and an integrated development environment (IDE), supporting more accessible and culturally relevant programming education.

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Programming languages have become one of the fundamental technologies driving the modern digital world. Almost every field, including education, healthcare, finance, business, communication, engineering, and scientific research, relies on software applications developed using programming languages. As technology continues to evolve, the demand for programmers and software developers is increasing rapidly across the world. Learning programming has therefore become an essential skill for students pursuing computer science and software engineering. However, despite its importance, programming remains difficult for many beginners because most programming languages use English-based syntax and keywords. Popular programming languages such as Java, Python, C++, C#, JavaScript, and PHP use English words for their keywords and commands. Statements such as `if`, `while`, `for`, `print`, `return`, and `class` are common in almost every programming language. While these keywords are easy to understand for individuals with strong English language skills, they create a significant obstacle for learners whose primary language is not English. Students often spend considerable time understanding the meaning of programming keywords instead of focusing on the logical concepts behind programming. As a result, beginners may find programming confusing, difficult, and discouraging during the early stages of learning. Pakistan is a multilingual country where many students receive their primary education in regional languages. Sindhi is one of the oldest and most widely spoken regional languages in Pakistan, particularly in the province of Sindh. Thousands of students in schools and universities communicate comfortably in Sindhi during their daily lives. However, when these students begin studying computer science, they encounter programming languages that require English proficiency. This sudden transition from native language learning to English-based programming often reduces students' confidence and makes it more challenging for them to understand basic programming [18], [19] [22]

concepts such as variables, loops, conditions, and expressions. The language barrier affects not only academic performance but also students' interest in software development. Many learners who possess strong logical thinking abilities struggle simply because they are unfamiliar with English programming terminology. Instead of concentrating on problem-solving techniques, they are forced to memorize foreign keywords before they can even write a simple program. Consequently, many talented students lose interest in programming despite having the capability to become successful software developers. This issue highlights the need for programming environments that support regional languages and provide a more accessible learning experience. Inspired by this idea, the present research introduces Latif Ji Boli, a Sindhi Programming Language Interpreter designed specifically for Sindhi-speaking learners. The project aims to provide an educational programming environment where users can write computer programs entirely in the Sindhi language while still learning the fundamental principles of programming. Instead of using English keywords, the language introduces equivalent Sindhi commands that are easier for native speakers to understand and remember. This approach allows learners to focus on programming logic rather than language translation. The proposed system has been developed using the Java programming language because Java provides a robust, object-oriented, and platform-independent environment suitable for compiler development. The system consists of multiple integrated components that work together to process Sindhi source code. These components include a Lexical Analyzer (Lexer), Token Management System, and Interpreter. Each component performs a specific function during program execution, ensuring that Sindhi source code is correctly analyzed and interpreted.

The lexical analyzer serves as the first stage of the compiler. It scans the source code character by character and divides the input into meaningful tokens. A token represents the smallest meaningful unit of a programming language, such as keywords, identifiers, numbers, operators, or string literals. The lexer implemented in this project recognizes Unicode Sindhi characters, enabling programmers to write source code naturally using [1], [2]

the Sindhi script. It also records the line number and column number of every token, allowing the system to generate meaningful error messages whenever invalid syntax

is detected. After lexical analysis, the generated tokens are passed to the token management component. This component stores important information about each token, including its type, value, line position, and column position. Proper token management enables the interpreter to distinguish between different categories of input elements, such as variables, operators, keywords, and literals. The structured representation of tokens simplifies program execution while improving error detection and debugging. The interpreter represents the execution engine of the programming language. Unlike traditional compilers that translate source code into machine code before execution, the interpreter directly evaluates the generated tokens and executes program statements one by one. The interpreter developed in this project supports variable declaration, assignment operations, mathematical expressions, string handling, conditional statements, while loops, do-while loops, logical operators, comparison operators, and output statements. During execution, it also manages memory for variables and performs type checking to ensure that numeric and string values are used correctly. When runtime errors occur, such as accessing undeclared variables or performing invalid mathematical operations, the interpreter generates descriptive error messages that help users locate and correct the problem.

Beyond its technical implementation, the project also contributes to educational innovation by encouraging programming education in regional languages. Students who are more comfortable reading and writing in Sindhi can understand programming concepts more effectively because they no longer need to translate English keywords mentally before understanding program logic. This localized learning environment has the potential to improve student engagement, increase confidence, and encourage more learners to pursue careers in software engineering and information technology.

The development of Latif Ji Boli demonstrates that programming languages are not limited to English syntax. By combining compiler design principles with native language computing, this project provides an innovative educational platform that promotes digital inclusion while preserving linguistic identity. The proposed system not only introduces a functional Sindhi programming language but also establishes a foundation for future research in regional language computing, compiler

construction, and multilingual software development. As technology becomes increasingly global, supporting indigenous languages in software engineering will play an important role in making programming education accessible to a wider population and reducing language-related barriers in computer science.

1.2 PROBLEM STATEMENT

Programming has become an essential component of modern education and technological development. However, the majority of programming languages available today are designed using English-based syntax and keywords. Languages such as Java, Python, C++, JavaScript, and C# require programmers to understand English commands before they can write and execute programs successfully. While this approach has become the international standard, it presents significant challenges for students whose primary language is not English.

In Sindh, many students complete their early education in the Sindhi language and use Sindhi as their primary means of communication. When these students begin learning programming at colleges and universities, they face two learning challenges simultaneously. First, they must understand the logical concepts of programming, including variables, loops, conditions, operators, and expressions. Second, they must memorize English programming keywords and syntax. This dual learning process increases cognitive load and often discourages beginners from continuing their programming education.

Although numerous programming languages exist today, there is currently no widely accepted programming language designed specifically for Sindhi speakers. The absence of regional-language programming environments limits accessibility for native Sindhi learners and creates unnecessary dependence on English. Many students possess excellent analytical and problem-solving abilities but struggle because they

are unfamiliar with English terminology. Consequently, they often develop misconceptions about programming, believing that it is inherently difficult when, in reality, much of the difficulty arises from the language barrier.

Another challenge is the lack of educational tools that support programming using the Sindhi script. Existing compilers and interpreters are designed primarily for

English- based languages and do not recognize Sindhi keywords or Unicode Sindhi characters as valid programming syntax. This limitation prevents learners from expressing programming logic in their native language and reduces opportunities for localized computing education. To overcome these challenges, this project proposes the development of a Sindhi Programming Language Interpreter named *Latif Ji Boli*. The proposed system allows programmers to write programs using Sindhi keywords while preserving the fundamental principles of programming. The compiler recognizes Sindhi syntax through lexical analysis, converts the source code into structured tokens, and executes the program using an interpreter. This approach enables students to learn programming concepts directly in their native language while reducing dependence on English terminology.

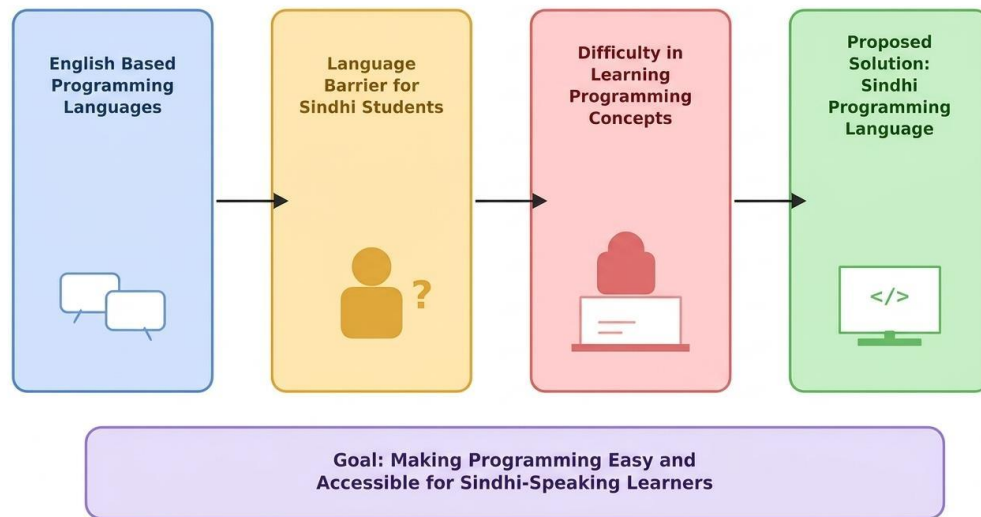


Figure 1.1: Research Problem Diagram

1.3 RESEARCH MOTIVATION

Technology has transformed almost every aspect of human life, making computer literacy and programming essential skills for the future workforce. Educational institutions around the world are continuously seeking innovative teaching methods that improve students' understanding of technical subjects. One effective approach is to teach complex concepts using the learner's native language. Research in educational psychology suggests that students learn more efficiently when new concepts are introduced using a familiar language because they can focus on

understanding ideas instead of translating terminology. This project was motivated by the observation that many Sindhi-speaking students experience unnecessary difficulty during introductory programming courses. During classroom discussions and laboratory sessions, students often spend more time asking about the meaning of English keywords than understanding programming algorithms. Simple commands such as `if`, `while`, `print`, and `return` become obstacles even though the underlying logical concepts are straightforward. This indicates that language itself has become a barrier to learning rather than programming.

Another motivation behind this research is the limited representation of regional languages in software engineering. While significant progress has been made in supporting regional languages for word processing, web applications, and mobile interfaces, comparatively little attention has been given to compiler construction and programming language development for indigenous languages. Developing a programming language based on Sindhi demonstrates that regional languages can also participate in advanced areas of computer science. The project also seeks to preserve and promote the Sindhi language in the digital age. As information technology continues to expand globally, there is increasing concern that many regional languages may become underrepresented in technological applications. Introducing Sindhi into programming environments encourages the use of the language in scientific and technical contexts, thereby contributing to its digital preservation [19], [23].

The motivation behind *Latif Ji Boli* is therefore educational, technological, and cultural. Educationally, it simplifies programming for beginners. Technologically, it demonstrates the practical implementation of a regional-language compiler and interpreter. Culturally, it promotes the Sindhi language within the field of software engineering and encourages greater participation of Sindhi-speaking students in computer science and technology.

1.4 RESEARCH AIM

The primary aim of this research is to design and develop a functional Sindhi Programming Language Interpreter that enables users to write, analyze, and execute computer programs using Sindhi language syntax instead of conventional English-

based programming keywords. The project aims to demonstrate that compiler construction techniques can successfully support regional languages through the implementation of lexical analysis, token generation, expression evaluation, variable management, control structures, and runtime execution. The developed system seeks to provide a practical educational platform where Sindhi-speaking students can understand programming concepts without first mastering English programming terminology.

Another important aim is to reduce the language barrier that exists in computer science education by providing a localized programming environment. Through Unicode support, Sindhi keyword recognition, structured token management, and interpreter-based execution, the system allows users to develop logical thinking skills while programming entirely in their native language. The project further aims to contribute to the growing field of regional-language computing by demonstrating that indigenous languages can be effectively integrated into modern software development technologies. By implementing the compiler using Java and designing modular components such as the Lexer, Token Management System, and Interpreter, the research provides a foundation for future improvements including syntax parsing, semantic analysis, integrated development environments, AI-assisted debugging, code optimization, and support for additional programming constructs.

Finally, the study aims to encourage greater participation of Sindhi-speaking students in software engineering by making programming education more inclusive, understandable, and culturally relevant, thereby supporting the broader objective of digital accessibility and technological empowerment through native-language computing.

1.5 OBJECTIVES

The primary objective of the proposed research is to design and implement a Sindhi Programming Language Interpreter that enables users to write and execute computer programs using the Sindhi language. The system aims to eliminate the language barrier that many Sindhi-speaking students experience when learning programming for the first time. By replacing English programming keywords with equivalent Sindhi keywords, the project allows learners to concentrate on programming logic

instead of memorizing unfamiliar terminology. Another important objective is to develop a lexical analyzer capable of recognizing Unicode Sindhi characters and converting Sindhi source code into meaningful tokens. The lexical analyzer serves as the first stage of the compiler and plays an essential role in identifying keywords, identifiers, numbers, operators, strings, and symbols according to the rules of the proposed language.

The project also aims to implement a token management system that stores token information, including token type, value, line number, and column number. This structured representation of tokens enables accurate interpretation of source code and provides meaningful syntax and runtime error messages whenever incorrect statements are encountered. The system further aims to support the essential programming constructs required by beginner programmers. These include variable declarations, assignment statements, numeric and string data types, arithmetic operations, relational operators, logical operators, conditional statements, while loops, and do-while loops.

Supporting these features ensures that users can solve practical programming problems using the Sindhi language while learning the same logical concepts taught in traditional programming languages. Another objective of this research is to promote regional language computing by demonstrating that compiler technologies can successfully support indigenous languages. The project encourages educational institutions to consider native-language programming environments as complementary learning tools for introductory programming courses.

Finally, the research aims to provide a strong foundation for future development. The modular architecture of the compiler allows future researchers to extend the language by adding arrays, functions, classes, object-oriented programming concepts, semantic analysis, optimization techniques, code generation, and integrated development environments.

The major objectives of the project can therefore be summarized as follows:

- To design a programming language using Sindhi keywords
- To implement a Unicode-compatible lexical analyzer
- To develop an efficient token management system

- To design an interpreter for executing Sindhi programs
- To support variables, arithmetic expressions, loops, and conditional statements
- To provide meaningful syntax and runtime error handling
- To simplify programming education for Sindhi-speaking students
- To promote regional language computing and digital inclusion
- To provide a foundation for future compiler research

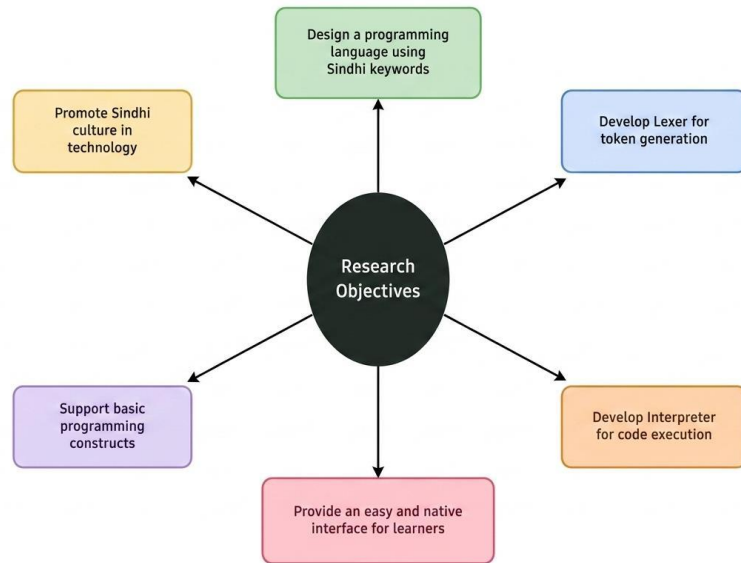


Figure 1.2: Research Objective Mind Map

1.6 SCOPE OF THE STUDY

The scope of this research is limited to the design and implementation of a compiler-interpreter system that supports programming using the Sindhi language. The project primarily focuses on developing the core components required for processing Sindhi source code, including lexical analysis, token generation, expression evaluation, and interpretation. The system is intended as an educational programming environment rather than a replacement for industrial programming languages such as Java, Python, or C++. The lexical analyzer forms the first stage of the system and is responsible for reading Sindhi source code and identifying different lexical elements. The lexer recognizes Sindhi keywords, identifiers, numbers, strings, operators, punctuation symbols, and comments. It also records the exact location of each token, allowing syntax errors to be reported with accurate line and column numbers.

The token management system stores the lexical information produced by the lexer. Each token contains a token type, token value, line number, and column number. This structured representation simplifies interpretation and provides better debugging information during program execution. The interpreter executes Sindhi source code directly without producing intermediate machine code. It supports declaration of numeric and string variables, assignment statements, mathematical calculations, string concatenation, relational operators, logical operators, conditional statements, while loops, do-while loops, and output statements. The interpreter also performs runtime validation to prevent invalid operations such as accessing undeclared variables or performing arithmetic on incompatible data types.

The proposed programming language currently supports two basic data types: numeric values and strings. These data types are sufficient for demonstrating compiler construction concepts and introductory programming techniques. More advanced structures such as arrays, user-defined functions, recursion, object-oriented programming, inheritance, and file handling are beyond the scope of the present research.

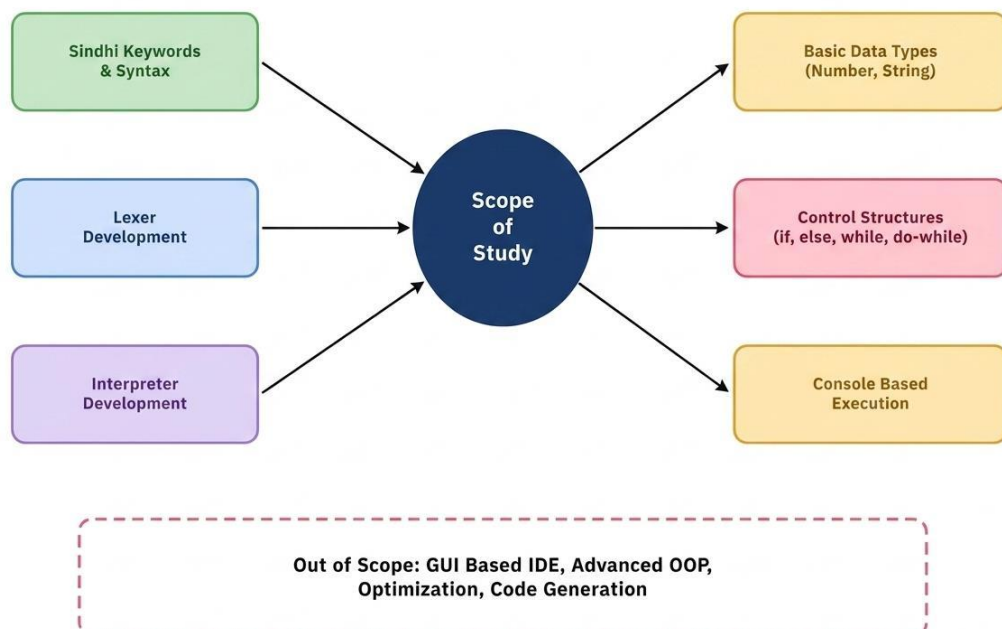


Figure 1.3: Scope Diagram

The implementation is developed using Java because of its portability, object-oriented architecture, and extensive support for compiler development. The backend compiler components communicate with the user interface through Spring Boot services, while Angular provides an interactive web-based programming environment where users can write Sindhi code and view execution results.

1.7 SYSTEM OVERVIEW

Step	Processing Stage	Description
1	Source Code Input	User writes Sindhi source code
2	Request Transfer	Angular sends source code to Spring Boot
3	Lexical Analysis	Lexer scans source code and generates tokens
4	Token Management	Tokens are stored and classified
5	Interpretation	Interpreter executes program statements
6	Output/Error Generation	Execution result or error is produced
7	Result Display	Frontend displays the result to the user

Table 1.4: Program Execution Flow

Component	Technology/Module	Primary Responsibility
Presentation Layer	Angular	Accept Sindhi source code and display results
Application Layer	Spring Boot	Receive requests and connect the interface with the interpreter
Lexical Analyzer	Java Lexer	Scan source code and generate tokens
Token Management	Token Management System	Store token type, value, line and column information
Interpreter	Java Interpreter	Evaluate statements and execute the program

Table 1.1: Major System Components

The proposed system, Latif Ji Boli – A Sindhi Programming Language Interpreter, follows a modular architecture in which each software component performs a specialized task during program execution. The architecture consists of four major layers: the presentation layer, application layer, compiler layer, and execution layer. The presentation layer provides an interactive interface through which users write Sindhi source code. The frontend is developed using Angular, allowing users to enter programs, submit source code, and view execution results in real time. The interface is designed to remain simple and user-friendly so that beginners can focus on programming rather than software complexity.

The application layer is implemented using Spring Boot. It acts as a communication bridge between the frontend and the compiler engine. Whenever a user submits Sindhi source code, the Spring Boot application receives the request and forwards the code to the compiler components for processing.

The compiler layer consists of three major modules implemented in Java:

- Sindhi Lexer
- Sindhi Token
- Sindhi Interpreter

The lexer scans Unicode Sindhi source code and converts it into tokens. These tokens represent keywords, identifiers, numbers, operators, strings, comments, and symbols recognized by the language grammar. The interpreter reads the token sequence and

executes each programming statement according to the grammar rules defined by the language. It manages variables, evaluates expressions, executes loops and conditional statements, performs arithmetic operations, and produces program output.

The execution flow of the proposed system can be summarized as follows:

1. User writes Sindhi source code
2. Angular sends the source code to Spring Boot
3. Spring Boot forwards the code to the Java compiler
4. The lexer converts source code into tokens
5. Tokens are stored and classified
6. The interpreter executes each statement
7. Program output or error messages are returned to the frontend
8. Results are displayed to the user

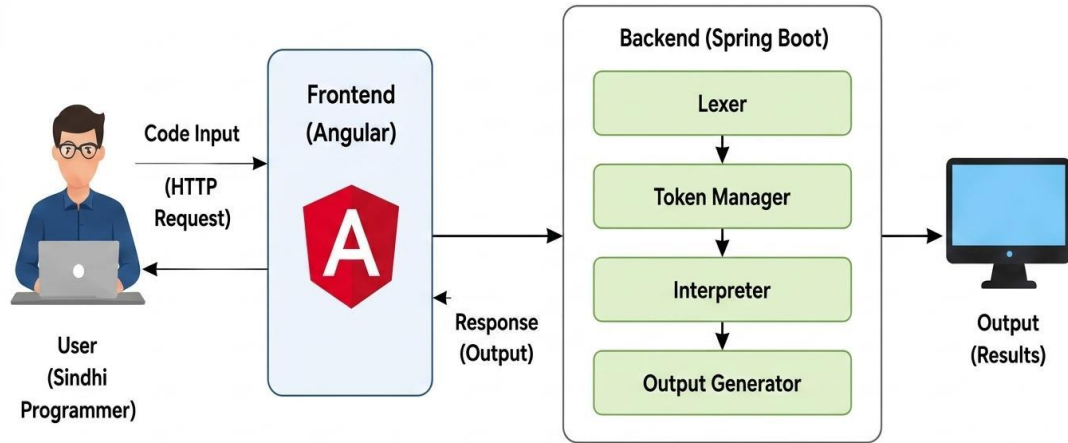


Figure 1.4: Overall System Architecture

1.8 FEATURES OF THE PROPOSED SYSTEM

Feature	Description
Unicode Sindhi Support	Recognizes Sindhi characters and keywords in Unicode text
Lexical Analysis	Identifies keywords, identifiers, numbers, strings, operators and symbols
Variables	Supports numeric and string variable declaration and assignment
Expression Evaluation	Supports arithmetic, relational and logical expressions
Control Structures	Supports conditional statements, while loops and do-while loops
String Handling	Supports string literals, output and concatenation
Error Handling	Reports syntax and runtime errors with line and column information

Table 1.2: Supported Programming Features

The Latif Ji Boli – Sindhi Programming Language Interpreter has been designed to provide a simple, efficient, and user-friendly programming environment for Sindhi-speaking learners. Unlike conventional programming languages that rely entirely on English keywords, the proposed system enables programmers to write source code using Sindhi language syntax while maintaining the logical structure of modern programming languages. The project combines compiler construction principles with regional language computing to produce an educational programming platform. One of the most important features of the system is its ability to recognize Unicode Sindhi characters. Since Sindhi uses a unique script that differs from the English alphabet, the compiler has been designed to process Unicode text correctly. The lexical

analyzer normalizes the source code and identifies Sindhi keywords without requiring any special keyboard layouts or language conversions. This allows users to write programs naturally in their native language.

Another major feature is the implementation of a Lexical Analyzer (Lexer). The lexer reads the Sindhi source code character by character and divides the program into meaningful lexical units called tokens. During lexical analysis, the system identifies keywords, identifiers, numbers, operators, strings, comments, and punctuation symbols according to the grammar rules defined for the language. The lexical analyzer serves as the first stage of the compiler and ensures that invalid characters or unsupported symbols are detected before interpretation begins. The Token Management System is another essential component of the project. Each token generated by the lexer contains detailed information, including the token type, token value, line number, and column number. This structured representation enables the interpreter to process the program accurately while providing meaningful syntax and runtime error messages. Recording the exact location of every token greatly simplifies debugging because users can quickly identify the location of mistakes within their source code.

The proposed language supports variable declaration using Sindhi keywords. Users can declare numeric variables as well as string variables before assigning values. Once variables have been declared, they can be updated, manipulated, and referenced throughout the program. The interpreter maintains a symbol table that stores variable names together with their current values and corresponding data types. This ensures consistency during program execution and prevents the use of undeclared variables. Another important feature is expression evaluation. The interpreter evaluates arithmetic expressions containing addition, subtraction, multiplication, division, and modulus operations. Parentheses may also be used to control operator precedence, allowing users to construct more complex mathematical expressions. In addition to arithmetic operations, the interpreter evaluates relational operators such as greater than, less than, greater than or equal to, less than or equal to, equality, and inequality. These operators are used extensively in decision-making statements and loops.

The language also supports logical operations using Sindhi equivalents of logical AND and logical OR. Logical expressions are evaluated by the interpreter to

determine whether specific conditions are true or false. This capability enables programmers to develop more sophisticated decision-making algorithms using multiple conditions within a single statement.

The project includes comprehensive support for conditional statements. The interpreter evaluates conditional expressions and executes different blocks of code depending on whether the specified condition evaluates to true or false. Conditional statements represent one of the most fundamental concepts in programming because they enable software to make intelligent decisions based on input values and runtime conditions. Looping constructs are also implemented within the proposed language. The compiler currently supports both while loops and do-while loops, allowing users to execute a sequence of statements repeatedly while a specified condition remains true. Looping is essential for solving repetitive computational problems efficiently and introduces students to one of the core concepts of algorithm development. Another valuable feature is string handling. The language supports string literals enclosed

within quotation marks and allows strings to be printed as program output. String concatenation enables textual information to be combined with variables and arithmetic expressions, making program output more informative and user-friendly.

The interpreter has also been designed with runtime error handling capabilities. During execution, the system detects common programming errors such as undeclared variables, incompatible data types, division by zero, invalid mathematical operations, and unsupported expressions. Instead of terminating unexpectedly, the interpreter produces descriptive error messages containing the corresponding line and column numbers. These detailed diagnostics significantly improve the learning experience because beginners can identify and correct mistakes without unnecessary confusion.

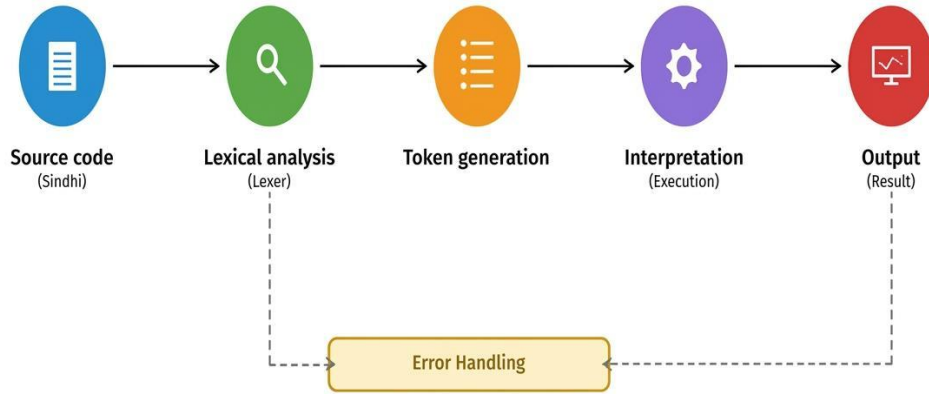


Figure 1.5: Compiler Working Flow

1.9 EXPECTED OUTCOMES

Data Type	Current Support	Scope
Numeric	Supported	Basic calculations and expressions
String	Supported	Text storage, output and concatenation
Arrays	Not supported	Future enhancement
Functions	Not supported	Future enhancement
Object-Oriented Features	Not supported	Outside current scope

Table 1.3: Data Types and Scope of the Proposed Language

The successful completion of this research is expected to produce a functional Sindhi Programming Language Interpreter capable of processing and executing source code written entirely in the Sindhi language. The system demonstrates that compiler construction techniques can successfully support regional languages while preserving standard programming principles.

The project is expected to improve programming education among Sindhi-speaking students by reducing dependence on English syntax and allowing learners to understand programming concepts more effectively. Students should be able to write variables, arithmetic expressions, conditions, loops, and output statements using Sindhi keywords while developing the same logical reasoning skills required for modern software development.

Another expected outcome is increased awareness regarding regional-language computing. Educational institutions and researchers may recognize the importance of integrating indigenous languages into software engineering education. This may encourage the development of additional programming languages for other regional languages spoken throughout Pakistan. The modular implementation of the compiler is also expected to facilitate future research. Because the lexer, token manager, and interpreter have been implemented as independent components, future developers can extend the language with additional compiler phases, advanced programming features, and integrated development environments.

Finally, the project is expected to demonstrate that linguistic diversity and technological advancement can coexist. By allowing programming in Sindhi, the research contributes simultaneously to educational accessibility, software engineering innovation, and the digital preservation of an important regional language.

1.10 CHAPTER SUMMARY

This chapter introduced the background and motivation behind the development of the Latif Ji Boli – Sindhi Programming Language Interpreter. The discussion highlighted the growing importance of programming education and the challenges faced by Sindhi-speaking students due to the dominance of English-based programming languages. The problem statement explained the lack of regional-language programming environments and justified the need for a compiler capable of processing Sindhi source code.

The chapter further presented the objectives, scope, and significance of the study while explaining how the proposed system contributes to programming education, compiler construction, and regional language computing. The architecture of the system was described, including the interaction between the Angular interface, Spring Boot backend, lexical analyzer, token management system, and interpreter. Important system features such as Unicode support, expression evaluation, control. The expected outcomes demonstrate that the proposed system can improve programming accessibility for Sindhi-speaking learners while promoting indigenous language computing within software engineering. Overall, this chapter establishes

the theoretical and practical foundation for the remaining chapters of the thesis. The next chapter presents the literature review, where existing research on programming languages, compiler construction, interpreters, lexical analysis, and regional-language computing is examined and compared with the proposed system.

CHAPTER 2

LITERATURE REVIEW

2.1 BACKGROUND

Programming languages are the fundamental tools used to communicate with computers and develop software applications. Every software application, whether it is a mobile application, web application, desktop software, operating system, or artificial intelligence system, is developed using one or more programming languages. Over the last several decades, programming languages have evolved significantly, becoming more powerful, flexible, and easier to use. Languages such as C, C++, Java, Python, JavaScript and C# dominate the software industry because they provide efficient methods for developing reliable and scalable applications. Despite these advancements, almost all widely used programming languages continue to rely on English-based syntax and keywords. The language barrier is particularly significant in developing countries where many students receive their early education in regional languages. In Pakistan, a large number of students communicate primarily in Urdu, Sindhi, Punjabi, Pashto, Balochi, and other regional languages before entering higher education. When these students begin studying software engineering or computer science, they are expected to learn programming entirely through English. This sudden transition often reduces learning efficiency and discourages students from pursuing careers in software development [1], [8].

Researchers have recognized this educational challenge and have proposed localized programming languages to improve accessibility. Native-language programming systems replace English programming keywords with equivalent words in local languages while maintaining the same logical structure of programming. Such systems enable students to focus on programming concepts rather than foreign-language terminology. Several studies have shown that programming in a familiar language [18], [19], [20]

improves comprehension, increases student confidence, and enhances classroom participation during introductory programming courses.

The Latif Ji Boli – Sindhi Programming Language Interpreter follows this educational philosophy by introducing a programming language specifically

designed for Sindhi-speaking learners. The system allows users to write programs using Sindhi keywords while preserving the logical principles of modern programming languages. Instead of translating programming concepts into English, students interact directly with programming logic using familiar Sindhi expressions. Another important contribution of the project is its support for the Unicode Sindhi script. Traditional compiler implementations are generally designed for ASCII or English text processing. The proposed system demonstrates that compiler technologies can successfully process Unicode-based regional languages without compromising execution accuracy. This capability broadens the applicability of compiler construction techniques beyond English-centric software systems.

This chapter reviews existing literature related to programming languages, compiler construction, lexical analysis, interpreters, token management, regional-language programming, and educational programming environments. The review establishes the theoretical foundation for the proposed research and identifies gaps that justify the development of a Sindhi Programming Language Interpreter.

2.2 EVOLUTION OF PROGRAMMING LANGUAGES

Programming languages have undergone continuous evolution since the invention of digital computers. Early computers were programmed directly using machine language, where programmers wrote binary instructions consisting of zeros and ones. Although machine language provided complete control over computer hardware, it was extremely difficult to write, understand, and maintain. Even simple programs required hundreds of binary instructions, making software development slow and error-prone. [1], [2]. [3]

To simplify programming, assembly language was introduced. Assembly language replaced binary instructions with symbolic mnemonics such as MOV, ADD, JMP, and SUB. Assemblers translated these symbolic instructions into machine code. Although assembly language significantly improved programmer productivity, it remained closely tied to specific processor architectures and still required detailed knowledge of computer hardware. [1].

The introduction of high-level programming languages marked a major milestone in software engineering. Languages such as FORTRAN, COBOL, BASIC, Pascal, and

C enabled programmers to express algorithms using human-readable syntax instead of hardware instructions. Compilers translated high-level source code into executable machine code, allowing software to be developed more efficiently while remaining portable across different computer systems. More recently, interpreted languages such as Python and JavaScript have become extremely popular because of their simplicity and flexibility. Python is widely used in artificial intelligence, machine learning, automation, and scientific computing, while JavaScript dominates web development. Although these languages simplify programming, they continue to rely on English keywords, creating accessibility challenges for non-English speakers. Modern programming languages have therefore become increasingly sophisticated, but they have not addressed the linguistic challenges faced by native-language learners. This limitation has motivated researchers to investigate localized programming languages that preserve computational logic while replacing English syntax with native-language keywords. The proposed Sindhi Programming Language follows this direction by providing an educational programming language that supports Unicode Sindhi keywords while implementing standard compiler design techniques. [1], [2].

2.3 COMPILER AND INTERPRETER TECHNOLOGIES

Compilers and interpreters are fundamental components of programming language implementation. Both are responsible for translating human-readable source code into instructions that computers can execute. Although they perform similar functions, their execution strategies differ significantly. A compiler translates an entire program into machine code before execution begins. After successful compilation, an executable program is generated that can be executed multiple times without recompilation. Programming languages such as C and C++ primarily rely on compiler-based execution because compiled programs generally provide higher execution speed and better optimization. [1], [2].

An interpreter, on the other hand, executes source code directly without producing a permanent executable file. Instead of translating the entire program at once, the interpreter reads individual statements, analyzes them, and immediately performs the corresponding operations. Languages such as Python and JavaScript rely heavily on

interpreter-based execution because it provides immediate feedback during software development. The proposed Sindhi Programming Language adopts an interpreter-based approach. This design decision was made because educational programming environments benefit from immediate execution and interactive debugging. Students receive instant feedback after writing their programs, allowing them to identify mistakes and understand programming behavior more effectively. The compiler architecture implemented in this project follows a modular design consisting of three primary components: the lexical analyzer, the token management system, and the interpreter. Each component performs a specialized function while cooperating to process Sindhi source code. [1], [2].

The lexical analyzer scans the input source code and generates tokens representing keywords, identifiers, operators, numbers, strings, comments, and punctuation symbols. These tokens are then stored by the token management system before being passed to the interpreter for execution. The interpreter evaluates arithmetic expressions, stores variables, executes conditions and loops, performs logical operations, and generates program output. This modular architecture closely resembles standard compiler design principles while adapting them specifically for Unicode Sindhi programming syntax. [1], [2].

2.4 LEXICAL ANALYSIS IN COMPILER DESIGN

Lexical analysis is the first phase of compiler construction and plays a vital role in translating source code into a form that can be processed by later stages of the compiler. The lexical analyzer, commonly referred to as the lexer or scanner, reads the source code character by character and groups individual characters into meaningful sequences called tokens. These tokens represent the smallest meaningful units of a programming language and include keywords, identifiers, literals, operators, punctuation symbols, and comments. The efficiency of a compiler largely depends on the performance of its lexical analyzer because every source program must pass through this stage before interpretation or compilation can begin. A properly designed lexer improves compiler accuracy by detecting invalid characters, recognizing reserved keywords, ignoring unnecessary whitespace, and reporting lexical errors at an early stage. [1], [2].

In the proposed Latif Ji Boli system, the lexical analyzer has been implemented using Java and customized specifically for the Sindhi language. Unlike conventional lexical analyzers that recognize only ASCII or English characters, the lexer developed for this research supports Unicode-based Sindhi script. It identifies Sindhi keywords, variable names, numeric literals, string literals, arithmetic operators, logical operators, relational operators, parentheses, punctuation symbols, and assignment operators according to the grammar rules of the proposed language. Another important responsibility of the lexer is maintaining the exact location of every token. During scanning, the system records the corresponding line number and column number for each token. This information enables the compiler to generate accurate syntax and runtime error messages whenever invalid input is encountered. For beginners,

descriptive error reporting significantly improves the learning process because mistakes can be corrected quickly without searching through the entire program. The lexer implemented in this project also ignores comments and unnecessary whitespace while preserving meaningful program structure. As a result, users can write well-formatted Sindhi programs without affecting compiler execution. This design closely follows the lexical analysis techniques used in modern programming languages while adapting them for regional-language computing. [1], [4], [12].

2.5 TOKENIZATION AND TOKEN MANAGEMENT

After the lexical analyzer recognizes the individual elements of the source code, the next stage involves converting these elements into structured tokens. Tokenization is the process of assigning each lexical element a predefined category that can be understood by the interpreter. Every token generated by the lexer contains important information regarding the corresponding program element. In the proposed compiler, each token consists of four major attributes: [1], [2].

- Token Type
- Token Value
- Line Number
- Column Number

The token type identifies the category of the lexical element. Examples include keyword, identifier, number, string literal, operator, delimiter, or comment. The token value stores the actual text recognized by the lexer, while the line number and column number specify the exact location where the token appears within the source program. The token management component serves as an intermediate layer between lexical analysis and interpretation. Instead of repeatedly scanning the source code, the interpreter processes the organized token sequence generated by the lexer. This separation of responsibilities improves software modularity and simplifies compiler maintenance. [1], [2].

For example, when a Sindhi variable declaration statement is entered, the lexer first recognizes the declaration keyword, variable name, assignment operator, and value. These individual elements are converted into separate tokens before being forwarded to the interpreter. The interpreter then examines the sequence of tokens and determines the corresponding execution behavior. [1].

The token management approach adopted in this project follows standard compiler construction techniques used in modern programming languages. However, unlike conventional systems that operate exclusively on English syntax, the proposed implementation successfully handles Sindhi Unicode keywords while maintaining identical compiler architecture. The use of structured tokens also improves debugging. Since each token contains its exact location within the source code, syntax errors can be reported with precision. Instead of displaying vague compiler messages, the system informs users of the specific line and column where an error occurred, allowing beginners to correct mistakes more efficiently. [1], [2].

2.6 UNICODE SUPPORT IN REGIONAL LANGUAGE PROGRAMMING

One of the major technical challenges associated with regional-language programming is the processing of non-English characters. Most traditional programming languages were originally designed around the ASCII character set, which contains only English letters, numbers, and punctuation symbols. Regional languages such as Sindhi require Unicode because their alphabets contain hundreds

of unique characters that cannot be represented using standard ASCII encoding. [4], [12].

Unicode is an international character encoding standard that assigns a unique numerical value to every character regardless of language or operating system. It enables computers to process multiple writing systems consistently while preserving character accuracy across different software platforms. The proposed Sindhi Programming Language relies entirely on Unicode processing. Every Sindhi keyword, identifier, and string literal is stored and processed using Unicode . This enables [21]

programmers to write source code naturally without converting Sindhi text into Roman script or using English equivalents. [4], [12].

Unicode support also improves software portability. Since Java provides excellent built-in Unicode support, the compiler can execute Sindhi programs consistently across different operating systems without requiring platform-specific modifications. This portability makes the proposed system suitable for educational institutions using different computing environments. Another important advantage of Unicode processing is its contribution to digital language preservation. As more software applications begin supporting regional languages, indigenous languages become increasingly represented within technological ecosystems. The proposed compiler therefore contributes not only to programming education but also to the broader objective of preserving Sindhi as a modern digital language. [4], [12].

2.7 REVIEW OF NATIVE LANGUAGE PROGRAMMING SYSTEMS

Several researchers have explored the concept of developing programming languages using native languages instead of English. These systems were primarily created to simplify introductory programming education and reduce linguistic barriers for beginners. Programming environments based on Arabic, Hindi, Chinese, Japanese, and Russian have demonstrated that students generally understand programming concepts more quickly when syntax is presented in their native language. Although the internal compiler architecture remains unchanged, replacing English keywords with localized equivalents significantly improves readability and learner confidence.

Educational programming languages such as Scratch, Blockly, and Logo also emphasize simplicity and accessibility. Although these systems are not fully regional- language programming languages, they demonstrate the importance of reducing complexity during introductory programming education. Their success indicates that beginner-friendly programming environments can improve computational thinking without overwhelming students with advanced syntax. [7].

The proposed Sindhi Programming Language follows similar educational principles while introducing complete Unicode Sindhi programming syntax. Unlike visual programming environments, the system allows learners to write textual source code using Sindhi keywords. Consequently, students simultaneously learn programming syntax and compiler behavior, preparing them for future study of industrial programming languages. Unlike previous educational programming environments, the proposed compiler also introduces practical compiler design concepts including lexical analysis, tokenization, interpreter execution, Unicode processing, variable management, expression evaluation, and runtime error handling. These features make the system suitable both as a programming language and as a compiler construction project. [7].

2.8 COMPARATIVE ANALYSIS OF EXISTING SYSTEMS

Several programming languages and educational programming environments are available for learning and software development. However, most mainstream languages use English-based keywords, while educational environments such as Scratch focus on simplifying programming through visual blocks rather than providing a text-based programming syntax in a regional language. Research on programming- language localization also shows that localization can involve different aspects of a language, including keywords and other elements of syntax.

Table 2.1: Comparative Analysis of C, Python, and Latif Ji Boli

Feature	C	Python	Latif Ji Boli
Text-Based Programming	Yes	Yes	Yes
English Keywords	Yes	Yes	No
Sindhi Keywords	No	No	Yes

Feature	C	Python	Latif Ji Boli
Unicode Support	Limited	Yes	Yes
Sindhi Unicode Syntax	No	No	Yes
Interpreter-Based Execution	No	Yes	Yes
Variables	Yes	Yes	Yes
Numeric Data Types	Yes	Yes	Yes
String Handling	Yes	Yes	Yes
Arithmetic Operations	Yes	Yes	Yes
Logical Operations	Yes	Yes	Yes

To identify the position of the proposed system, Latif Ji Boli is compared with commonly used programming languages and an educational programming environment. The comparison focuses on language syntax, Unicode support, execution approach, educational orientation, and regional-language support. [16]

The comparison shows that C, Python and Latif Ji Boli provide extensive programming capabilities but rely primarily on English-based language syntax. Scratch provides a beginner-friendly educational environment, but it uses a visual block-based approach rather than a text-based programming language using Sindhi syntax. In contrast, Latif Ji Boli combines text-based programming, Sindhi keywords, Unicode Sindhi script, and interpreter-based execution. Therefore, the proposed system addresses a specific gap by providing a programming environment designed for Sindhi-speaking beginners while retaining fundamental programming constructs found in established languages.

2.9 RESEARCH GAP

The literature review indicates that considerable research has been conducted on compiler construction, lexical analysis, interpreters, and educational programming languages. Similarly, several native-language programming initiatives have been developed for languages such as Arabic, Hindi, Chinese, and Japanese. However, very limited research has focused on programming language development for the Sindhi language. Existing programming languages continue to rely almost entirely on

English syntax, creating learning difficulties for beginners who possess limited English proficiency. Although compiler technologies are well established, few implementations have attempted to integrate Unicode Sindhi processing with lexical analysis and interpreter execution within a complete programming environment. [1], [4], [7].

The proposed Latif Ji Boli project addresses this research gap by introducing a complete Sindhi Programming Language Interpreter capable of processing Unicode Sindhi source code while implementing standard compiler design principles. This contribution extends existing research in both compiler construction and regional-language computing. [1], [4].

2.10 CHAPTER SUMMARY

This chapter reviewed the theoretical concepts and previous studies related to programming languages, compiler construction, lexical analysis, interpreters, Unicode processing, and regional-language programming systems. The discussion highlighted the evolution of programming languages from machine language to modern high-level languages while emphasizing the continued dominance of English-based programming syntax. A comparative analysis showed that current programming languages do not adequately support Sindhi-language programming despite increasing interest in regional-language computing. Finally, the research gap identified the absence of a comprehensive Sindhi Programming Language Interpreter that combines Unicode processing, lexical analysis, token management, interpreter execution, and educational programming features.

The findings presented in this literature review provide the theoretical foundation for the next chapter, which explains the research methodology, system design, architecture, implementation process, and technologies used in developing the proposed Latif Ji Boli – Sindhi Programming Language Interpreter.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 BACKGROUND

The research methodology is an important part of any software engineering project. It explains the steps that are followed to develop a system. This includes how the problem was found how the system requirements were analyzed what technologies were chosen, how the software was designed and implemented and how the final system was tested and evaluated. A good methodology makes sure that the development process is organized, reliable and can be repeated while producing a high-quality software product.

The goal of this research is to develop Latif Ji Boli, a Sindhi Programming Language Interpreter. This system allows users to write and execute computer programs using the Sindhi language. Unlike programming languages that use English syntax the proposed system uses Sindhi keywords while keeping the logical structure of modern programming languages. The methodology used for this project combines compiler construction principles with software engineering practices to produce an educational programming environment. The development process has stages. First the project requirements were identified by analyzing programming language features and compiler construction techniques. After that the overall system architecture was designed by dividing the project into modules such as the lexical analyzer, token management system, interpreter, expression evaluator and error handling component. Each module does a job while interacting with other modules to process Sindhi source code efficiently. Java was chosen as the implementation language because of its object-oriented architecture, portability and extensive Unicode support. The backend services were implemented using Spring Boot while Angular was used to develop the graphical user interface. These technologies provide a programming environment where users can write Sindhi programs execute them and get immediate results.

3.2 RESEARCH METHODOLOGY

Stage	Research Activity	Purpose
1	Problem Identification	Identify language-related barriers in programming education
2	Literature Review	Study compiler construction, interpreters, Unicode and localized programming
3	Requirement Definition	Define functional and non-functional requirements
4	System Design	Design modular architecture and processing components
5	Implementation	Develop lexer, token management and interpreter modules
6	Testing and Evaluation	Verify correctness and runtime behavior
7	Documentation	Document findings, limitations and future work

Table 3.1: Research Methodology Stages

The research methodology used for this project follows a software engineering approach that combines theoretical research with practical implementation. The methodology focuses on solving the problem created by English-based programming languages through the development of a compiler-interpreter system that supports Sindhi programming syntax. The research started by identifying the challenges faced by Sindhi-speaking students while learning programming. Existing programming languages such as Java, Python and C++ require learners to understand keywords before they can write programs. This language dependency motivated the development of a localized programming environment where programming concepts could be expressed using the Sindhi language.

After identifying the research problem a detailed literature review was conducted to study compiler construction techniques, lexical analysis, interpreter design, Unicode processing, educational programming languages and regional-language computing. This review provided the foundation necessary for designing the proposed compiler architecture. The implementation phase focused on transforming concepts into a working software system. Individual modules were developed independently before being integrated into a compiler-interpreter framework. The modular development strategy improved maintainability, simplified debugging. Allowed each component to be tested independently.

Throughout the project emphasis was placed on software quality, code readability, modular architecture and user accessibility. The final implementation demonstrates that compiler technologies can successfully support languages without compromising programming functionality.

The Latif Ji Boli system is a Sindhi Programming Language Interpreter that enables users to write and execute computer programs using the Sindhi language. The system uses Sindhi keywords. Supports Unicode characters. The development of the Latif Ji Boli system is a step towards providing a localized programming environment for Sindhi-speaking students.

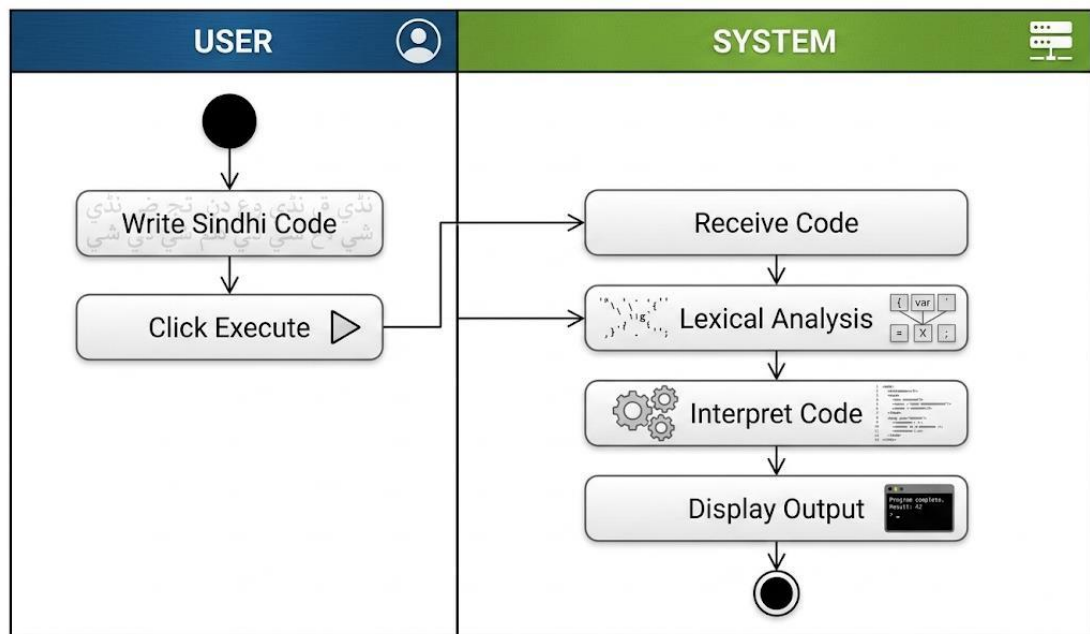


Figure 3.1: Activity Diagram

3.3 RESEARCH DESIGN: [5], [6].

The research design used in this project follows the principles of Design Science Research (DSR). Design Science Research is commonly used in software engineering because it focuses on creating technological solutions to practical problems. Of merely studying an existing phenomenon DSR emphasizes designing, implementing and evaluating an artifact that addresses a specific research problem. [11]

In this project the artifact is the Latif Ji Boli system. The research design consists of sequential stages that ensure systematic development. The first stage involved

identifying the challenges associated with English-based programming languages. The second stage focused on defining system requirements based on compiler construction concepts and programming language fundamentals. During the stage the compiler architecture was designed, including the lexer, token manager, interpreter and

execution engine. The fourth stage involved implementation using Java, Spring Boot and Angular. Finally extensive testing was performed to verify correctness, reliability and usability.

The research design also emphasizes modularity. Of developing the compiler as a single monolithic application separate modules were created for lexical analysis, token management, interpretation and runtime execution. This design simplifies maintenance while enabling expansion of the programming language.

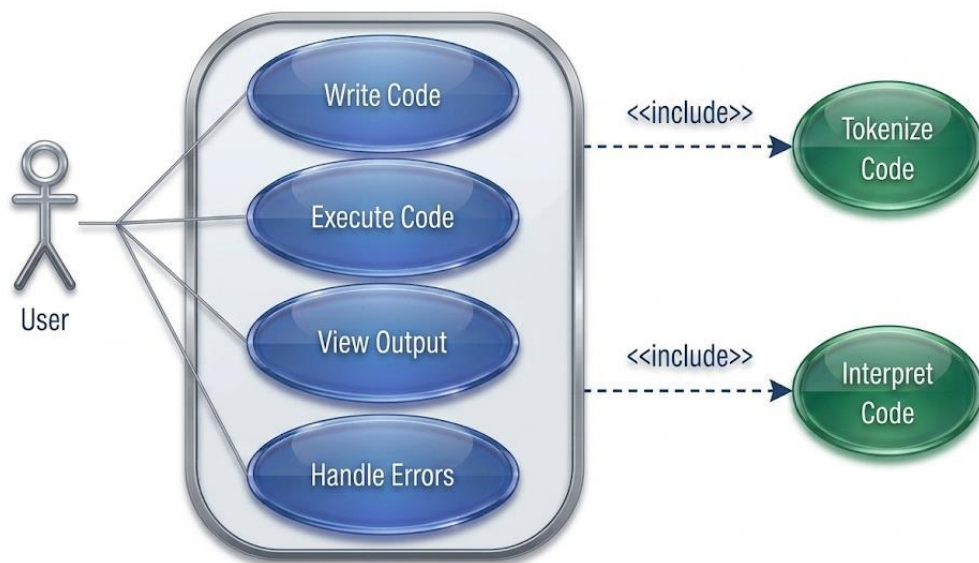


Figure 3.2: Use Case Diagram

3.4 REQUIREMENT ANALYSIS

Requirement analysis is the foundation of every software engineering project. Before implementation begins it is essential to determine the non-functional requirements of the proposed system. Requirement analysis ensures that the developed software satisfies user needs while remaining technically feasible.

The functional requirements of the proposed Latif Ji Boli system were identified based on the objectives of the project. The system must allow users to write source code entirely in the Sindhi language. It must recognize Sindhi keywords process Unicode characters correctly and execute programming statements according to predefined language rules.

The Latif Ji Boli system should support declaration and assignment enabling users to create numeric and string variables. Arithmetic operations including addition, subtraction, multiplication, division and modulus must be available for calculations. The system must also evaluate logical expressions to support decision-making statements. Conditional statements represent another functional requirement. The Latif Ji Boli system should correctly evaluate if- conditions and execute the appropriate code block depending on the result of the logical expression. Similarly looping constructs such as while and do-while loops should repeatedly execute statements while specified conditions remain true.

The system must also provide output through print statements. Users should be able to display text messages, variable values and arithmetic results on the output console. Error handling constitutes another functional requirement. The Latif Ji Boli system must detect syntax, undeclared variables, incompatible data types and unsupported operations. Whenever an error occurs the system should generate error messages together with the corresponding line and column numbers.

3.5 SOFTWARE DEVELOPMENT METHODOLOGY

The development of the proposed Latif Ji Boli system follows the Incremental Software Development Model. This methodology was selected because compiler construction naturally consists of independent components that can be developed and tested separately before integration. During the increment the lexical analyzer was implemented. The lexer was responsible for scanning Sindhi source code recognizing Unicode characters identifying keywords, operators, identifiers, numbers, strings and punctuation symbols and converting them into tokens.

The second increment focused on developing the management system. This component stores token information, including type, value, line number and column number. Proper token management simplifies. Improves compiler maintainability.

The third increment involved implementing the interpreter. The interpreter evaluates expressions executes variable declarations performs assignments manages program memory processes conditional statements and controls loop execution.

The fourth increment concentrated on integrating all modules with the Spring Boot backend and Angular frontend. This integration enabled users to submit Sindhi programs through a web interface while receiving execution results immediately after interpretation. The final increment consisted of testing, debugging and optimization. Individual modules were first tested independently using unit testing principles before complete system integration was performed. After integration several Sindhi programs containing operations, loops, conditional statements and string manipulation were executed to verify compiler correctness.

3.6 SYSTEM DESIGN

System design is a phase of software development. It transforms the requirements identified during the analysis stage into a software architecture. A designed system ensures that every software component performs its assigned responsibilities while interacting efficiently with other components. For the proposed Latif Ji Boli system a modular architecture was adopted to simplify development, testing, maintenance and future expansion.

The proposed system consists of two sections: the user interface and the compiler engine. The user interface provides an environment where programmers can write Sindhi source code and execute programs. The compiler engine processes the submitted source code through stages including lexical analysis, token generation, interpretation, expression evaluation and output generation. The compiler engine itself is divided into independent modules. The first module is the Lexical Analyzer (Lexer)

which scans the Sindhi source code character by character and converts it into tokens. The second module is the Token Management System, which stores token information and passes the sequence to the interpreter. The third module is the Interpreter for executing statements evaluating expressions maintaining variables and generating the final output. The modular design provides advantages. Each module performs a responsibility reducing software complexity and improving

maintainability. Since the modules communicate through defined interfaces future developers can replace or improve one module without significantly affecting the remaining components. This design also simplifies debugging because software defects can be isolated within modules instead of affecting the entire system.

The Latif Ji Boli system is designed to be efficient, scalable and maintainable. The system uses an architecture, which makes it easy to add new features or modify existing ones. The system is also designed to be user-friendly with an intuitive interface that makes it easy for users to write and execute Sindhi programs.

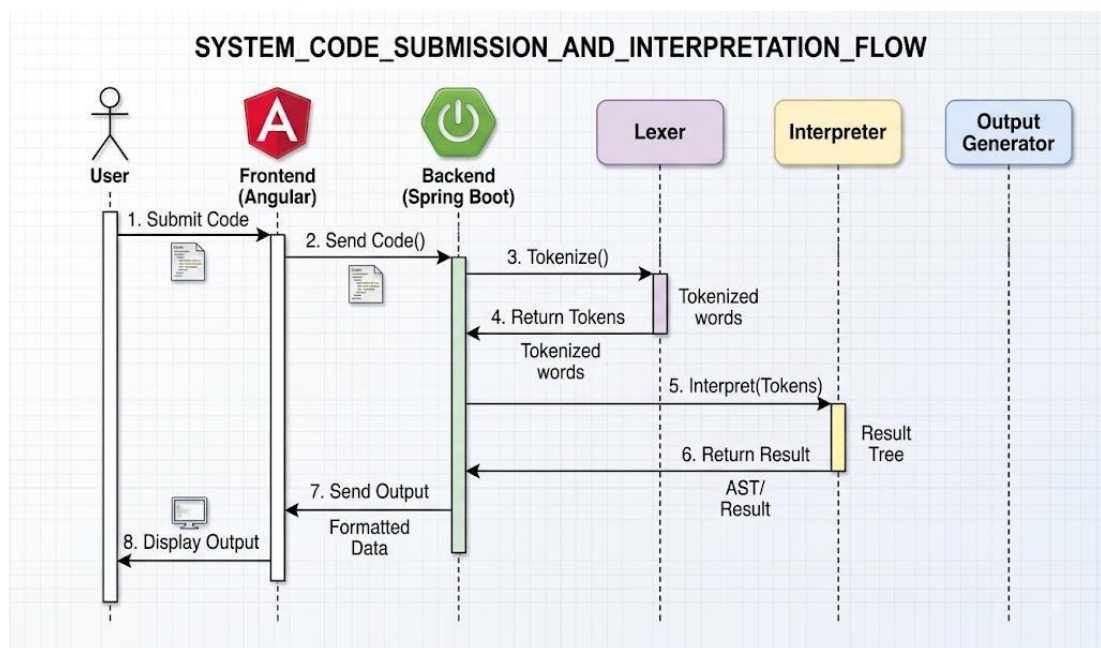


Figure 3.3: Sequence Diagram

3.7 SYSTEM ARCHITECTURE

The architecture of the proposed system consists of multiple software layers that cooperate to process Sindhi source code. Each layer performs a specialized function while interacting with adjacent layers to produce the final execution result.

The first layer is the Presentation Layer, developed using Angular. This layer provides an interactive graphical interface where users write Sindhi programs, submit source code, and observe execution results. The interface is intentionally designed to remain simple so that beginner programmers can focus on learning programming concepts rather than understanding complicated software environments. The second layer is the Application Layer, implemented using Spring Boot. The backend receives source code from the frontend and forwards it to the compiler engine for

processing. It also manages communication between the compiler modules and the graphical interface by returning execution results or compiler error messages. The third layer is the Compiler Layer, which contains the lexical analyzer, token management system, and interpreter. This layer performs the primary computational tasks required to process Sindhi programs.

The lexical analyzer scans the source code and recognizes programming constructs such as keywords, identifiers, numbers, operators, punctuation symbols, and string literals. Every recognized element is converted into a token containing detailed information regarding its type and location. The token management module organizes the generated tokens into a structured sequence that can be processed efficiently by the interpreter. Since every token stores its corresponding line number and column number, compiler errors can be reported accurately whenever invalid syntax is encountered.

The interpreter forms the final stage of the compiler architecture. It evaluates expressions, executes assignment statements, manages variables, performs arithmetic calculations, processes logical conditions, controls loops, and displays program output. The interpreter also performs runtime validation to prevent invalid operations and provides meaningful error messages whenever execution fails. The layered architecture improves software reliability by separating user interaction from compiler implementation. This separation also simplifies future enhancement because new programming language features can be incorporated within the compiler without modifying the user interface.

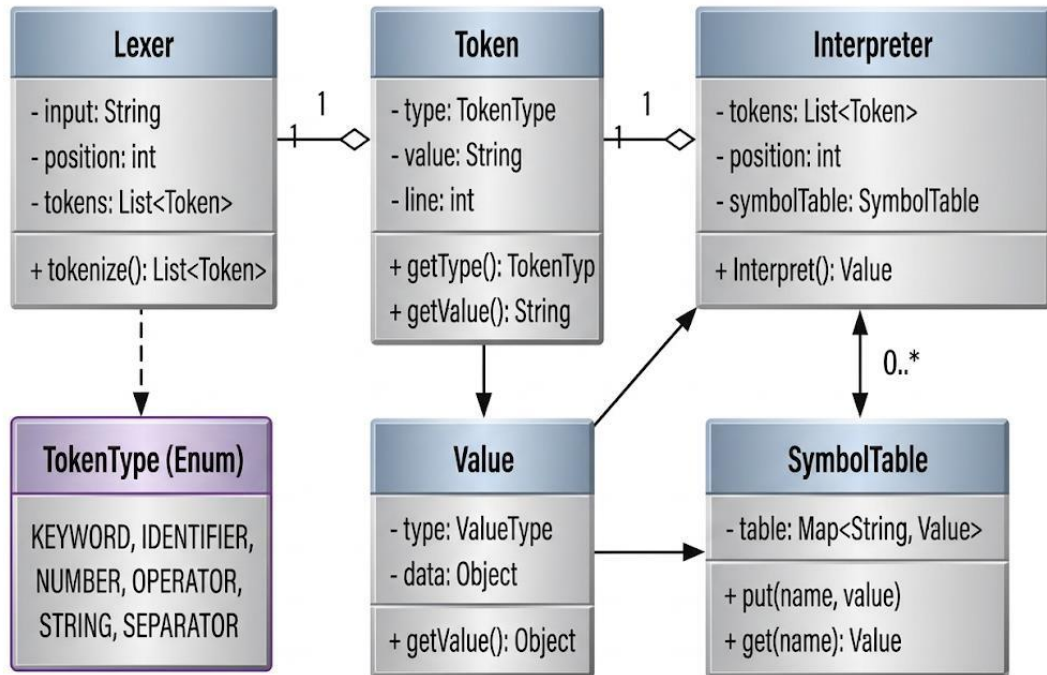


Figure 3.4: Class Diagram

3.8 IMPLEMENTATION METHODOLOGY

The implementation phase transformed the system design into a fully functional software application. Java was selected as the primary implementation language because it provides object-oriented programming features, strong Unicode support, portability, and an extensive collection of standard libraries suitable for compiler construction. [17].

The implementation process began with the development of the lexical analyzer. Regular expressions and character scanning techniques were used to recognize Sindhi keywords, identifiers, operators, numbers, strings, comments, and punctuation symbols. Since the programming language supports Unicode Sindhi characters, the lexer was carefully designed to process regional-language text accurately. Following lexical analysis, the token management component was implemented. Each token was represented as an object containing its category, textual value, line number, and column number. This structured representation simplifies communication between the lexer and interpreter while improving software readability. The interpreter was developed after completing the lexical analyzer. It processes the token sequence generated by the lexer and executes program statements according to the language

grammar. The interpreter maintains a symbol table that stores variable names together with their corresponding values and data types throughout program execution.

Arithmetic expressions were implemented using operator precedence rules to ensure accurate mathematical evaluation. Logical expressions were similarly evaluated before executing conditional statements and loops. Runtime memory management enables variables to retain their values during program execution while preventing undeclared variable access. The backend implementation was completed using Spring Boot, allowing communication between the compiler engine and the Angular frontend. Whenever users submit Sindhi source code, the backend invokes the compiler modules and returns execution results immediately after interpretation.

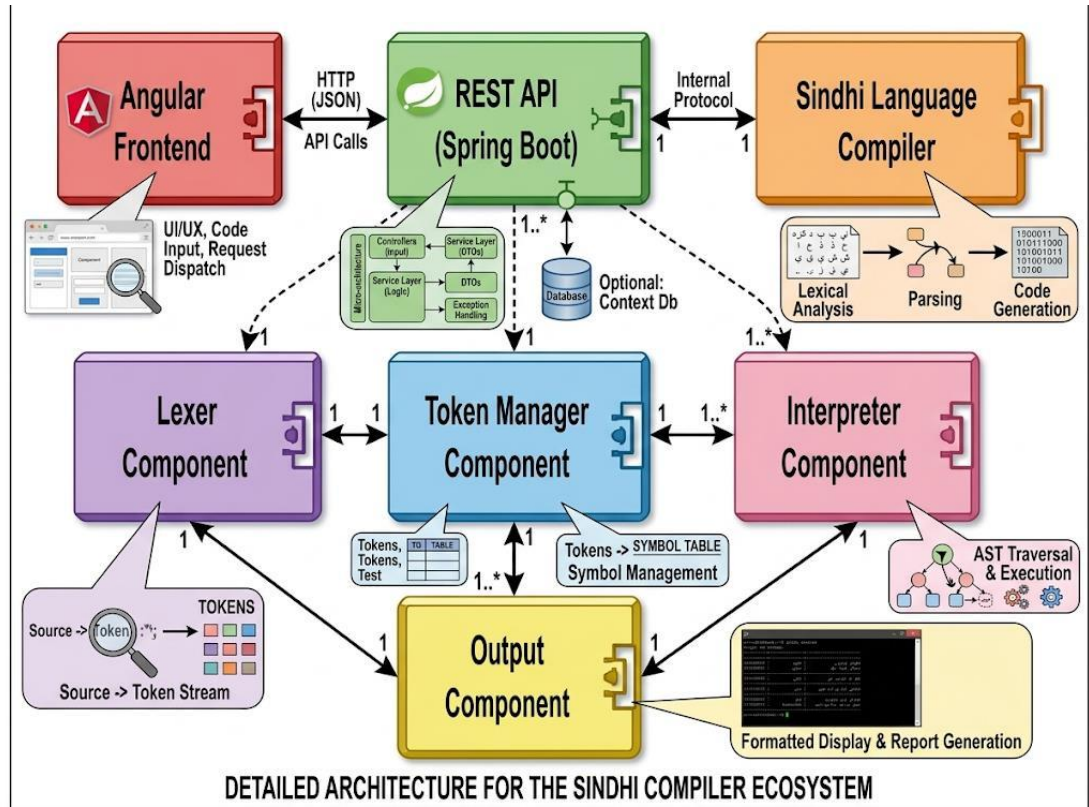


Figure 3.5: Component Diagram

3.9 TESTING METHODOLOGY

Software testing is an essential activity in software engineering because it verifies that the implemented system satisfies its specified requirements. The objective of testing is to identify software defects before deployment while ensuring that all system components operate correctly under different conditions. The proposed

compiler was tested using several categories of Sindhi source programs. Initially, lexical analysis was verified by supplying valid and invalid input containing keywords, identifiers, numbers, operators, strings, and unsupported characters. The correctness of token generation was examined by comparing generated tokens with expected compiler output. Interpreter testing focused on verifying program execution. Various arithmetic expressions were executed to confirm correct mathematical calculations. Variable declaration and assignment statements were tested to ensure accurate storage and retrieval of variable values. Conditional statements containing different logical [9], [10]

expressions were executed to verify decision-making behavior. Similarly, while loops and do-while loops were tested using different iteration counts to ensure proper loop execution. String processing functionality was verified by executing programs containing Sindhi text, concatenation operations, and output statements. Unicode compatibility was also tested by entering different Sindhi keywords and variable names.

Error handling constituted another major testing category. Invalid variable references, syntax mistakes, unsupported expressions, incompatible data types, and incorrect arithmetic operations were intentionally introduced into test programs. The compiler successfully detected these errors and produced descriptive messages together with line and column numbers. Both unit testing and integration testing were performed. Individual compiler modules were tested independently before complete system integration. After integration, end-to-end testing confirmed that communication between Angular, Spring Boot, the lexer, token manager, and interpreter functioned correctly.

3.10 TOOLS AND TECHNOLOGIES USED

Tool/Technology	Role in the Project
Java	Implementation of lexer, token management and interpreter
Spring Boot	Backend service layer and request handling
Angular	Web-based programming interface
Unicode	Representation and processing of Sindhi script
Git/GitHub	Source-code version control and collaboration
Postman	API request testing during development

Table 3.2: Tools and Technologies Used

Several software tools and technologies were used during the development of the proposed system. Each technology was selected according to its suitability for compiler construction and web-based software development. Java served as the primary programming language because of its object-oriented architecture, platform independence, Unicode support, and comprehensive standard libraries. [12], [13], [14], [15].

Spring Boot was selected for backend development because it simplifies REST API creation and enables efficient communication between the compiler engine and the frontend application. Angular was used to develop the graphical user interface. Angular provides a responsive and interactive environment where users can enter Sindhi programs and immediately observe execution results. [14], [15].

The Lexical Analyzer was implemented using Java character processing techniques together with regular expressions for keyword recognition and token generation. The Token Management System organizes lexical information into structured objects that improve compiler maintainability and simplify interpreter implementation. [12].

The Interpreter executes Sindhi source code directly by evaluating expressions, maintaining variables, processing conditions, and controlling loops. Unicode support was integrated throughout the system to ensure accurate processing of Sindhi script across different operating systems and browsers.

PROPOSED SYSTEM ARCHITECTURE: SINDHI COMPILER ECOSYSTEM

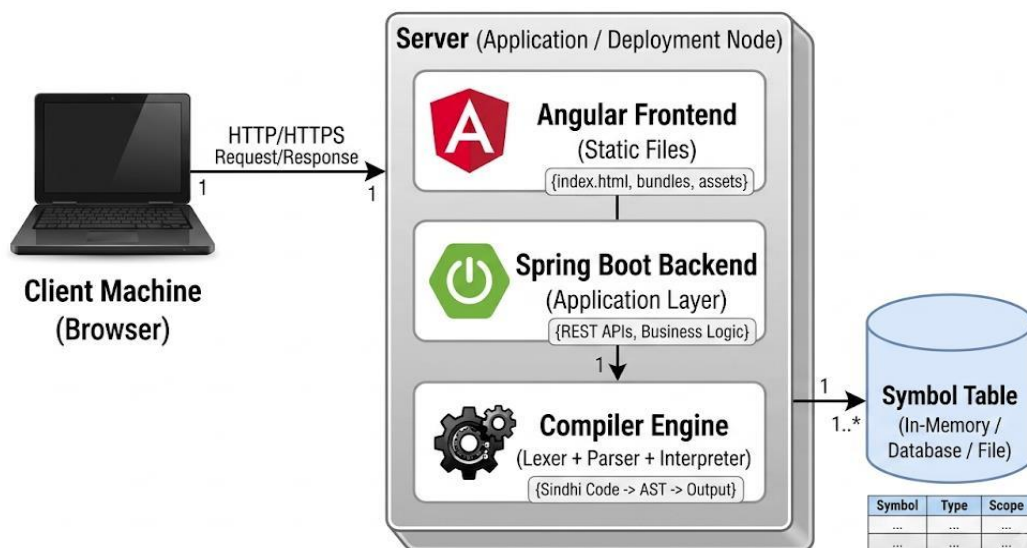


Figure 3.6: Development Diagram

3.11 WORKING METHODOLOGY OF THE PROPOSED SYSTEM: [1], [2].

The working methodology of the proposed compiler follows a sequence of well-defined processing stages. Initially, the programmer writes source code using Sindhi keywords through the Angular interface. After the user submits the program, the source code is transmitted to the Spring Boot backend for processing.

The backend forwards the source code to the lexical analyzer. The lexer scans each character, recognizes valid lexical elements, and converts them into structured tokens. The generated tokens are then stored by the token management component before being forwarded to the interpreter. The interpreter processes the token sequence sequentially while evaluating expressions, managing variables, executing loops, processing conditional statements, and generating output.

If syntax errors or runtime errors occur during execution, descriptive error messages are generated and returned to the frontend. Otherwise, the successful execution result is displayed to the user.

The entire execution process follows the sequence below:

9. User writes Sindhi source code
10. Angular sends the code to Spring Boot
11. Spring Boot invokes the compiler
12. Lexer performs lexical analysis
13. Tokens are generated
14. Interpreter executes the program
15. Output or error messages are generated
16. Results are displayed to the user

This workflow closely resembles modern compiler architecture while supporting native Sindhi programming syntax.

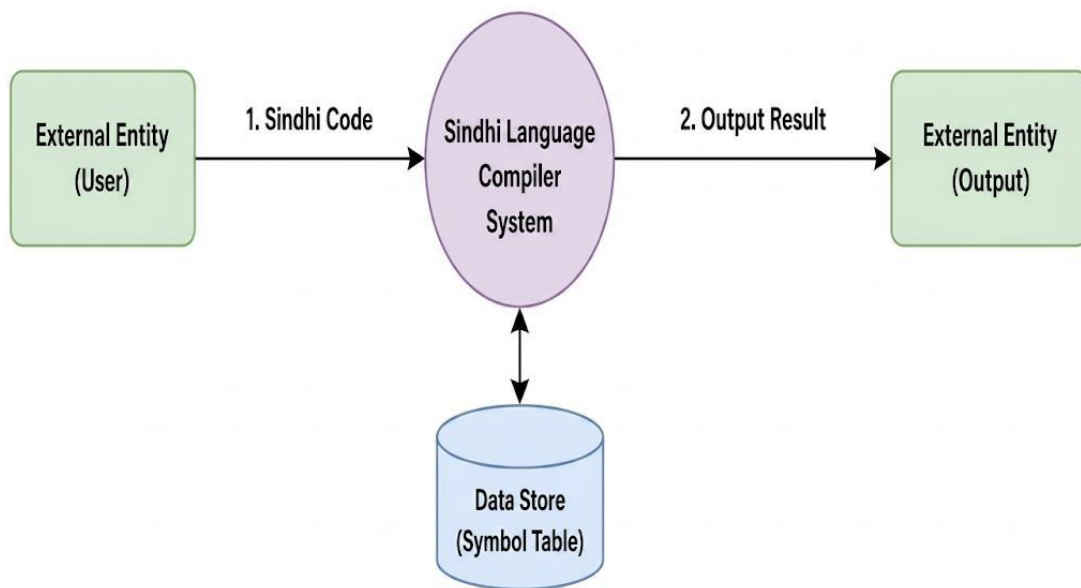


Figure 3.7: Data Flow Diagram (Level-0)

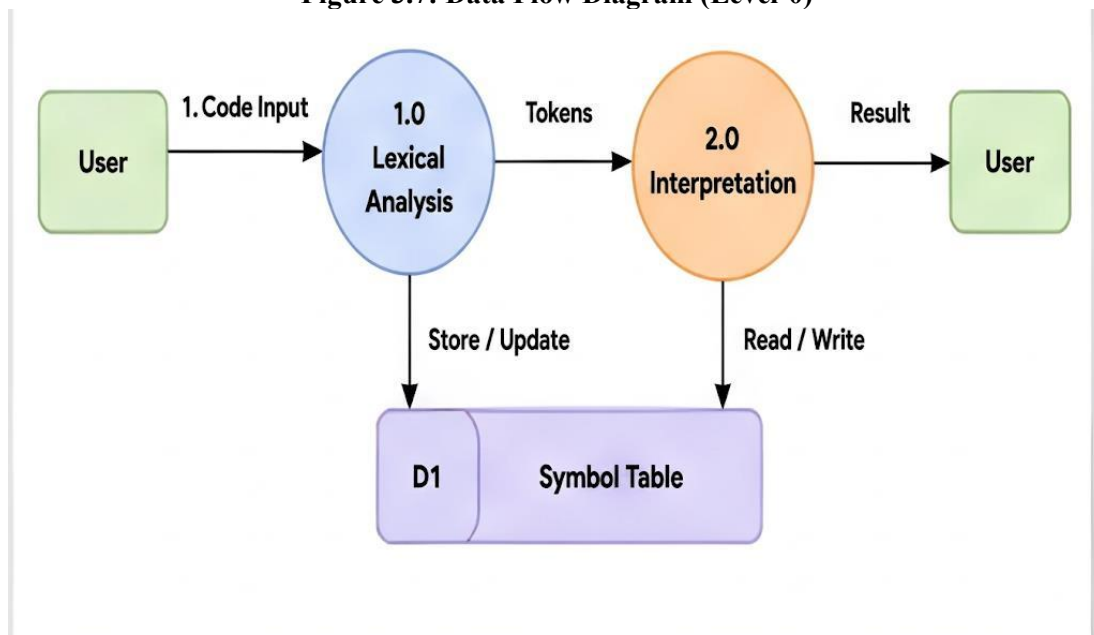


Figure 3.8: Data Flow Diagram (Level-1)

3.12 ADVANTAGES OF THE SELECTED METHODOLOGY

The methodology adopted for this research offers several significant advantages. First, the modular architecture simplifies software development by separating lexical analysis, token management, interpretation, and user interaction into independent modules. Second, the use of incremental software development reduced implementation complexity because each module could be completed and verified

before integrating it with the remaining system components. Third, Unicode support enables natural Sindhi programming without requiring language conversion or custom encoding techniques. Another important advantage is educational accessibility. Since the compiler supports Sindhi programming syntax, beginners can concentrate on programming logic rather than memorizing English keywords.

The selected methodology also improves software maintainability. Future developers can easily extend the language by introducing new programming constructs such as arrays, functions, recursion, classes, semantic analysis, optimization, or machine-code generation. Finally, the methodology successfully combines compiler construction theory with practical software engineering principles, producing a reliable educational programming environment suitable for Sindhi-speaking learners.

3.13 CHAPTER SUMMARY

This chapter described the research methodology adopted for developing the Latif Ji Boli – Sindhi Programming Language Interpreter. The discussion explained the research design, requirement analysis, software development methodology, system architecture, implementation strategy, testing procedures, and technologies used during software development.

The chapter demonstrated how compiler construction concepts such as lexical analysis, token generation, interpretation, Unicode processing, and runtime error handling were

integrated into a modular software architecture. The implementation methodology emphasized maintainability, portability, educational usability, and future extensibility. The testing methodology confirmed that the developed compiler correctly processes Sindhi source code, executes programming statements, and generates accurate output while detecting syntax and runtime errors effectively.

The next chapter presents the System Implementation and Results, where the internal implementation of the lexer, token management system, interpreter, user interface, and execution examples are explained in detail together with screenshots and experimental results.

CHAPTER 4

RESULTS AND DISCUSSION

4.1 INTRODUCTION

This chapter presents the implementation results and performance evaluation of the proposed Latif Ji Boli – Sindhi Programming Language Interpreter. The objective of this chapter is to demonstrate how effectively the developed system processes Sindhi-language source code and executes different programming constructs. The chapter evaluates the performance of each major module of the compiler, including the lexical analyzer, token management system, interpreter, expression evaluator, and user interface.

The development of a programming language involves more than simply recognizing keywords. Every source program passes through several processing stages before execution. The lexer scans the input program and converts the source code into meaningful tokens. These tokens are then organized by the token management module and passed to the interpreter. The interpreter evaluates expressions, stores variables, executes loops and conditional statements, and finally generates the required output. The successful interaction between these components demonstrates the correctness of the proposed compiler architecture.

The results obtained during implementation indicate that the system successfully accepts Sindhi-language source code written using Unicode characters. Unlike traditional programming languages that rely on English keywords, the proposed compiler recognizes Sindhi keywords while preserving the logical principles of programming. Various programs were executed to verify the correctness of arithmetic operations, conditional statements, loops, string handling, variable management, and runtime error detection.

The performance of the developed system was evaluated using different test cases representing common programming problems. Each test case was executed multiple times to verify consistency and correctness. The generated output confirmed that the compiler successfully interpreted Sindhi programming statements and produced accurate execution results.

In addition to functional correctness, the system was also evaluated in terms of usability. The Angular-based user interface allowed programmers to write Sindhi programs easily, while the Spring Boot backend efficiently communicated with the compiler engine. This architecture provides an educational programming environment where beginner programmers can understand programming concepts without depending entirely on English programming syntax. The following sections discuss the implementation results of each compiler module in detail together with an evaluation of the overall system performance.

4.2 SYSTEM IMPLEMENTATION RESULTS

After completing the implementation phase, the Sindhi Programming Language Interpreter were tested using several programming examples covering the major features supported by the language. These experiments verified the correctness of lexical analysis, token generation, interpreter execution, Unicode processing, and runtime error handling.

The implementation successfully integrated all software modules into a complete programming environment. The Angular frontend allows users to write Sindhi source code, while the Spring Boot backend forwards the program to the compiler engine. The lexer processes the source code, generates tokens, and sends them to the interpreter for execution. Finally, the interpreter evaluates the program and returns either the execution output or descriptive error messages.

The successful integration of these modules demonstrates that the proposed compiler architecture functions correctly and provides a practical educational programming environment.

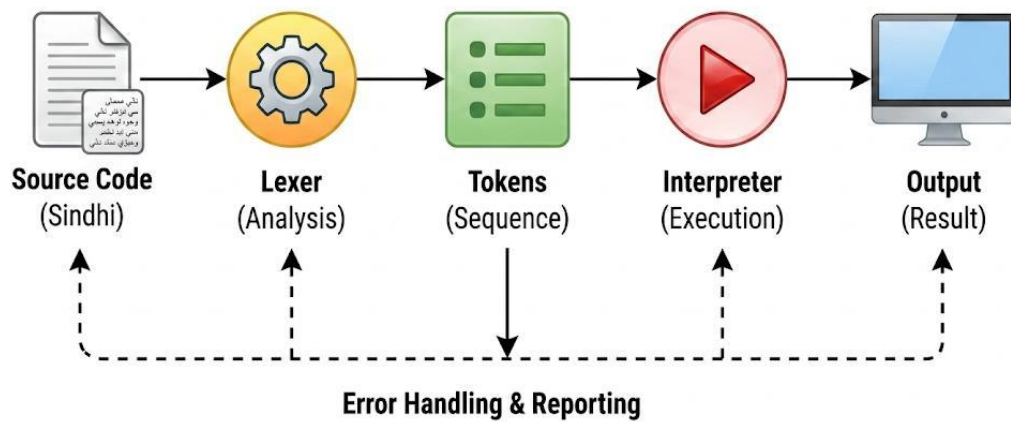


Figure 4.1: Compiler Pipeline Diagram

4.2.1 Lexical Analysis Results

The lexical analyzer represents the first stage of the compiler and was extensively tested using different Sindhi programming statements. During execution, the lexer successfully recognized Sindhi keywords, identifiers, operators, numeric values, string literals, punctuation symbols, and comments.

The lexer correctly ignored unnecessary whitespace while preserving the logical structure of the source program. Invalid characters were detected immediately, preventing incorrect programs from reaching later compiler stages. The implementation also demonstrated accurate Unicode processing. Sindhi keywords written using Unicode characters were recognized consistently without requiring transliteration into English. This confirms that Java's Unicode support, combined with the implemented lexical analysis techniques, provides reliable processing of regional- language programming syntax.

The generated tokens included complete information regarding token type, value, line number, and column number. This information greatly improved compiler diagnostics by enabling precise error reporting during interpretation.

Observed Results

- Sindhi keywords were recognized correctly
- Variable names were identified successfully
- Numeric and string literals were processed accurately

- Unicode Sindhi text was handled without encoding errors
- Invalid symbols generated lexical error messages

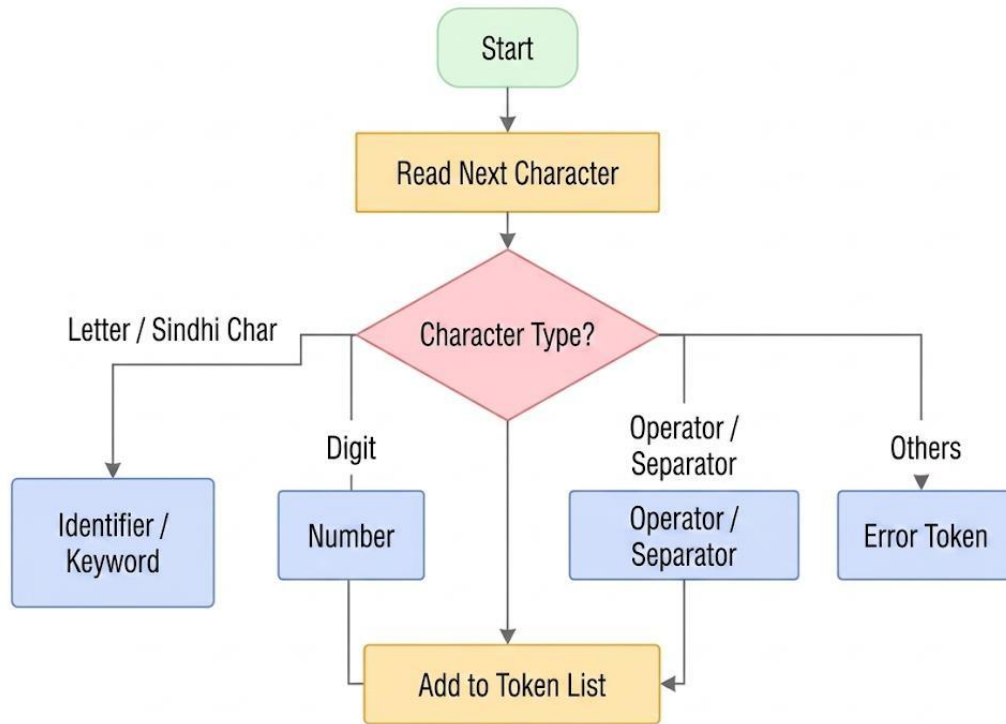


Figure 4.2: Lexer Internal Flowchart

4.2.2 Variable Declaration and Assignment

Variable management forms one of the most important components of any programming language. The developed interpreter was tested using several programs containing numeric and string variables. During execution, the interpreter successfully stored variables within its symbol table and maintained their values throughout program execution. The implementation also ensured that variable values remained consistent throughout program execution. Whenever variables participated in arithmetic expressions or conditional statements, their current values were retrieved correctly from the symbol table.

Observed Results

- Numeric variables were declared successfully
- String variables stored Unicode Sindhi text correctly
- Assignment statements executed without errors

- Symbol table management functioned correctly
- Undeclared variables generated descriptive runtime errors

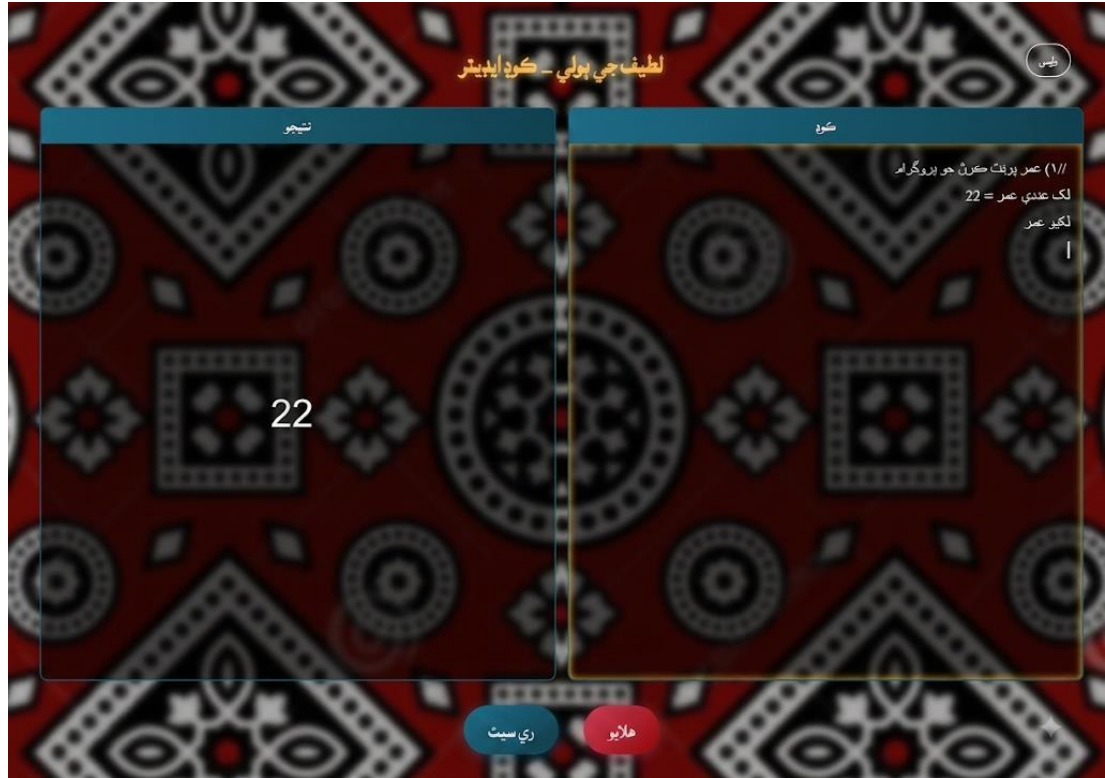


Figure 4.3: Variable Declaration and Assignment Output

4.2.3 Arithmetic Expression Evaluation

Mathematical calculations were performed to verify the correctness of the interpreter's expression evaluation module. Test programs included addition, subtraction, multiplication, division, modulus operations, and expressions containing multiple operators. The interpreter correctly followed operator precedence during expression evaluation. Parenthesized expressions also produced expected mathematical results, confirming that complex arithmetic calculations were evaluated correctly. Division by zero was tested intentionally to verify runtime validation. Instead of terminating unexpectedly, the interpreter generated an informative runtime error message explaining the nature of the problem.

These results demonstrate that the mathematical evaluation component of the interpreter performs reliably under both normal and exceptional conditions.



Figure 4.4: Arithmetic Expression Evaluation Output

4.2.4 Conditional Statement Execution

Conditional statements were implemented to enable decision-making within Sindhi programs. The interpreter evaluated relational expressions before determining which program block should be executed. Different comparison operators were tested using positive, negative, and equal values. Both true and false execution paths produced correct results. Nested conditional statements were also executed successfully, demonstrating that the interpreter can process multiple decision levels correctly. Logical operators combining multiple conditions were verified using different combinations of true and false values. The interpreter consistently evaluated logical expressions before executing the appropriate program statements. These experiments confirm that the conditional execution mechanism functions according to standard programming language semantics while using Sindhi keywords instead of English syntax.

Observed Results

- Simple conditions executed correctly
- Nested conditions produced expected results

- Logical expressions were evaluated accurately
- Comparison operators functioned correctly



Figure 4.5: Conditional Statement Execution Output

4.2.5 Loop Execution

Loop execution was evaluated using programs containing both while and do-while loops. Different iteration counts were selected to verify correct loop control under various conditions. The interpreter repeatedly executed loop bodies until termination conditions became false. Variable updates inside loop bodies were correctly reflected during subsequent iterations. This confirms that the interpreter maintains program state consistently throughout repetitive execution. The do-while implementation also demonstrated correct behavior by executing the loop body at least once before evaluating the termination condition. Several nested loop examples were also executed to verify interpreter stability. The generated outputs confirmed that nested looping structures performed as expected.

Observed Results

- While loops repeated correctly
- Do-while loops executed successfully

- Loop variables updated properly
- Nested loops functioned correctly



Figure 4.6: Loop Execution Output

4.2.6 String Handling

String processing is an essential feature of every programming language because it enables programmers to manipulate textual information in addition to numeric data. The proposed Latif Ji Boli – Sindhi Programming Language Interpreter supports string variables and Unicode-based Sindhi text. During implementation, the interpreter was tested with different programs containing Sindhi sentences, messages, and variable values to verify the correctness of string processing. The interpreter successfully recognized string literals enclosed within quotation marks and stored them in memory as string variables. During execution, the stored values were retrieved correctly and displayed through the output statements. Since the compiler is fully Unicode-compatible, Sindhi characters were processed accurately without encoding problems or character corruption.

Observed Results

- Sindhi string literals were recognized correctly

- Unicode characters were displayed without encoding errors
- String variables were stored and retrieved successfully
- Output statements displayed correct textual information



Figure 4.7: String Handling Output

4.2.7 Error Handling and Runtime Validation

One of the major objectives of the proposed compiler was to provide meaningful error messages that assist beginner programmers during software development. Unlike traditional compiler implementations that often generate complex diagnostic information, the proposed interpreter focuses on producing understandable error messages together with accurate program locations. During testing, several invalid programs were intentionally executed to evaluate the robustness of the error handling module. These test cases included undeclared variables, invalid assignment statements, unsupported operators, incorrect expressions, missing symbols, and division by zero. Whenever an invalid statement was encountered, the interpreter immediately terminated execution of the affected statement and generated a descriptive runtime message explaining the nature of the problem. Since every token

stores its corresponding line number and column number, the compiler accurately identified the location of each error within the source program.

The lexer also contributed to error handling by rejecting invalid Unicode characters and unsupported lexical elements before interpretation began. Consequently, syntax errors were detected during lexical analysis, while logical execution errors were identified by the interpreter. This two-stage validation process significantly improves software reliability because programming mistakes are detected as early as possible. For educational purposes, descriptive diagnostics also help beginners understand compiler behavior and correct programming mistakes more efficiently.

Observed Results

- Undeclared variables were detected successfully
- Invalid expressions generated descriptive error messages
- Division by zero was prevented
- Syntax errors displayed correct line and column numbers
- Unsupported tokens were rejected during lexical analysis

4.3 DISCUSSION OF RESULTS

Error/Validation Scenario	Expected System Behavior
Undeclared variable	Generate a descriptive runtime error with location information
Invalid expression	Reject the expression and report an error
Division by zero	Prevent invalid arithmetic operation
Syntax error	Report the error with line and column information
Unsupported token	Reject the token during lexical analysis
Incompatible data type	Report the invalid operation instead of continuing execution

Table 4.2: Error Handling and Validation Scenarios

The implementation results demonstrate that the proposed Latif Ji Boli – Sindhi Programming Language Interpreter successfully achieved the objectives established during the planning and design phases of the research. The completed system integrates lexical analysis, token management, interpreter execution, Unicode processing, and runtime validation into a single educational programming environment. The lexical analyzer performed accurately by recognizing Sindhi keywords, identifiers, operators, numbers, punctuation symbols, and string literals. The generated token sequence provided the interpreter with sufficient information for

correct program execution while simultaneously improving compiler diagnostics through precise token location information.

The interpreter also performed reliably during testing. Variables were created successfully, expressions were evaluated correctly, conditional statements followed the appropriate execution paths, and loops repeated according to their specified conditions. These results demonstrate that the interpreter correctly implements the logical behavior expected from a modern programming language while using Sindhi syntax instead of conventional English keywords. Unicode compatibility proved to be one of the most significant achievements of the project. Since the Sindhi language contains numerous characters unavailable within the ASCII character set, reliable Unicode support was essential. The successful execution of Sindhi programs confirms that Java provides an appropriate platform for regional-language compiler construction.

The project also demonstrates that compiler construction principles remain independent of human language. Lexical analysis, tokenization, interpretation, expression evaluation, and runtime validation operate according to identical computational principles regardless of whether the programming syntax is written in English or Sindhi. This finding supports the broader concept of multilingual programming environments. Overall, the implementation results confirm that the proposed compiler provides both a functional programming language and an effective educational platform for introducing compiler construction and programming concepts to Sindhi-speaking students.

4.4 FUNCTIONAL EVALUATION OF THE SYSTEM

The developed Latif Ji Boli interpreter was evaluated using test programs covering the main programming features supported by the language. The evaluation focused on the correctness of lexical analysis, program execution, Unicode handling, variable management, error detection, and output generation.

The lexical analyzer was tested with valid and invalid Sindhi source code. It correctly identified keywords, identifiers, numbers, strings, operators, and symbols, while invalid lexical structures were reported as errors. The interpreter was then tested with variable assignments, arithmetic and logical expressions, conditional

statements, loops, and string operations. The tested programs produced the expected results, and variable values were correctly maintained during execution.

Unicode support was also evaluated by executing programs containing Sindhi keywords and Unicode string values. The system correctly processed the Sindhi characters without corruption. Error handling was tested using invalid programs, and the interpreter successfully reported syntax and runtime errors along with relevant line and column information. These features help users identify and correct errors during program development.

The main evaluation results are summarized in Table 4.1.

Table 4.1: Functional Evaluation of Latif Ji Boli

Evaluation Criterion	Result
Lexical Analysis	Successful
Token Generation	Successful
Sindhi Unicode Support	Successful
Variable Management	Successful
Arithmetic Operations	Successful
Logical Operations	Successful
Conditional Statements	Successful
Loop Execution	Successful
String Handling	Successful
Error Detection	Successful
Output Generation	Successful

The evaluation shows that Latif Ji Boli successfully performs the fundamental programming operations implemented in the language and correctly processes Sindhi- based source code. The results demonstrate that the interpreter meets the main functional requirements defined for the project and provides a suitable foundation for learning basic programming concepts through Sindhi syntax.

4.5 ADVANTAGES OF THE DEVELOPED SYSTEM

The completed system offers several educational and technical advantages. The most important advantage is the reduction of language barriers in programming education. Students who primarily communicate in Sindhi can understand programming concepts more easily because programming keywords are expressed in their native language. Another advantage is the implementation of a complete compiler-interpreter framework rather than a simple syntax translator. The project includes lexical analysis, structured token generation, Unicode processing, expression evaluation, runtime

execution, and comprehensive error handling. Consequently, the compiler demonstrates genuine compiler construction techniques rather than simple keyword replacement.

The modular architecture represents another important benefit. Independent implementation of the lexer, token manager, interpreter, frontend, and backend improves software maintainability while facilitating future expansion. The use of Java, Spring Boot, and Angular also contributes to software portability, allowing the compiler to execute consistently across different computing environments.

Finally, the project promotes regional-language computing by demonstrating that indigenous languages can participate effectively within modern software engineering.

4.6 LIMITATIONS OF THE DEVELOPED SYSTEM

Although the developed compiler successfully meets its primary objectives, several limitations remain. The current implementation focuses primarily on introductory programming concepts and therefore supports only the programming constructs required by beginner programmers. Advanced language features such as arrays, user-defined functions, recursion, object-oriented programming, inheritance, exception handling, file processing, and database connectivity have not yet been implemented. These features may be incorporated in future versions of the compiler.

The current implementation also relies on interpreter-based execution rather than generating optimized machine code. While interpretation is appropriate for educational environments, compiled execution would provide improved performance for larger software systems. Similarly, syntax analysis currently emphasizes the implemented language grammar without incorporating advanced compiler optimization techniques or intermediate code generation. Despite these limitations, the current implementation provides a strong foundation upon which future researchers can extend the programming language.

4.7 CHAPTER SUMMARY

This chapter presented the implementation results and performance evaluation of the Latif Ji Boli – Sindhi Programming Language Interpreter. The discussion demonstrated that the developed system successfully processes Sindhi- language source code through lexical analysis, token generation, interpretation, Unicode processing, and runtime execution. Different programming constructs, including variable declarations, arithmetic expressions, conditional statements, loops, string handling, and error detection, were tested successfully. The evaluation confirmed that the compiler produces correct execution results while providing meaningful diagnostic information whenever programming errors occur.

The results also demonstrate that programming languages can successfully be implemented using regional languages without changing the underlying computational principles of compiler construction. By supporting Sindhi programming syntax, the proposed system reduces language barriers, promotes educational accessibility, and contributes to the growing field of multilingual software engineering.

Overall, the implementation confirms that the project successfully achieved its objectives and provides a reliable educational programming environment for Sindhi-speaking learners while establishing a strong foundation for future research in compiler construction and regional-language computing.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 BACKGROUND

This chapter presents the conclusion of the research conducted for the development of Latif Ji Boli – A Sindhi Programming Language Interpreter. It summarizes the objectives achieved, discusses the overall contribution of the project, highlights its significance in programming education, and presents recommendations for future improvements. The chapter also outlines potential research directions that can further enhance the proposed system and extend its capabilities beyond its current implementation.

The primary purpose of this project was to reduce the language barrier faced by Sindhi-speaking students when learning programming. Traditional programming languages use English keywords and syntax, making it difficult for beginners who are more comfortable with the Sindhi language. The proposed system demonstrates that compiler construction techniques can be successfully adapted to support regional languages while preserving the logical principles of programming.

5.2 CONCLUSION

The Latif Ji Boli – Sindhi Programming Language Interpreter was successfully designed, implemented, and evaluated during this research. The project achieved its primary objective of providing a programming environment where users can write and execute programs using Sindhi-language keywords instead of conventional English programming syntax. The developed system integrates several important compiler components, including lexical analysis, token management, expression evaluation, interpreter execution, Unicode processing, and runtime error handling. Each module was implemented independently and then integrated into a complete software system capable of processing Sindhi-language source code from input to execution.

The lexical analyzer successfully recognizes Unicode Sindhi characters and converts the input program into meaningful tokens such as keywords, identifiers, operators, numeric literals, string literals, and punctuation symbols. The generated tokens are stored by the token management component before being forwarded to the interpreter

for execution. The interpreter correctly executes variable declarations, assignment statements, arithmetic expressions, logical expressions, conditional statements, while loops, do-while loops, and output statements. During execution, the interpreter maintains variable values using a symbol table while generating meaningful runtime error messages whenever invalid operations occur.

The implementation also demonstrates that Unicode-based regional-language programming is technically feasible using Java. Throughout testing, Sindhi keywords were processed successfully without character encoding issues, proving that modern compiler technologies can effectively support indigenous languages. Another significant achievement of this research is the educational value of the proposed programming language. Beginner programmers often struggle because they must simultaneously learn programming concepts and English programming syntax. By replacing English keywords with Sindhi equivalents, the proposed language allows learners to focus on computational thinking and problem-solving rather than language translation. This approach has the potential to improve student confidence and make introductory programming courses more accessible.

Overall, the successful implementation confirms that programming languages are not limited to English syntax. The underlying computational principles remain identical regardless of the natural language used to express programming instructions. Therefore, the proposed system represents an important contribution toward regional- language computing, educational accessibility, and the digital promotion of the Sindhi language.

5.3 ACHIEVEMENT OF RESEARCH OBJECTIVES

At the beginning of this research, several objectives were established to guide the development of the proposed system. The implementation and testing results confirm that these objectives have been successfully achieved.

The first objective was to design a programming language that supports Sindhi-language syntax. This objective was accomplished through the development of a compiler capable of recognizing Unicode Sindhi keywords and programming constructs. The second objective involved implementing a lexical analyzer capable of processing Sindhi source code. The lexer successfully recognizes keywords,

identifiers, operators, strings, numeric values, and punctuation symbols while generating structured tokens. Another objective was to implement an interpreter capable of executing Sindhi-language programs. The interpreter correctly performs expression evaluation, variable management, conditional execution, loop execution, and runtime validation. The project also aimed to reduce language barriers in programming education. The use of Sindhi keywords allows beginner programmers to understand programming concepts without depending entirely on English syntax. Finally, the project sought to demonstrate that compiler construction techniques can support regional languages. The completed implementation successfully validates this objective through a functional compiler-interpreter system developed entirely using Java technologies.

5.4 CONTRIBUTIONS OF THE RESEARCH

The research makes several important contributions to software engineering and programming education.

One major contribution of this research is the development of an educational programming environment specifically designed around Sindhi programming syntax. Unlike conventional programming languages that require English syntax, the proposed system demonstrates that programming concepts can be taught using regional-language keywords without changing the logical structure of algorithms. The project also contributes to compiler construction by providing a practical implementation of lexical analysis, token management, interpreter execution, Unicode processing, and runtime error handling within a single software system.

From a cultural perspective, the project promotes the digital use of the Sindhi language by introducing it into an area traditionally dominated by English. This supports long-term language preservation while demonstrating that indigenous languages can participate in advanced technological applications.

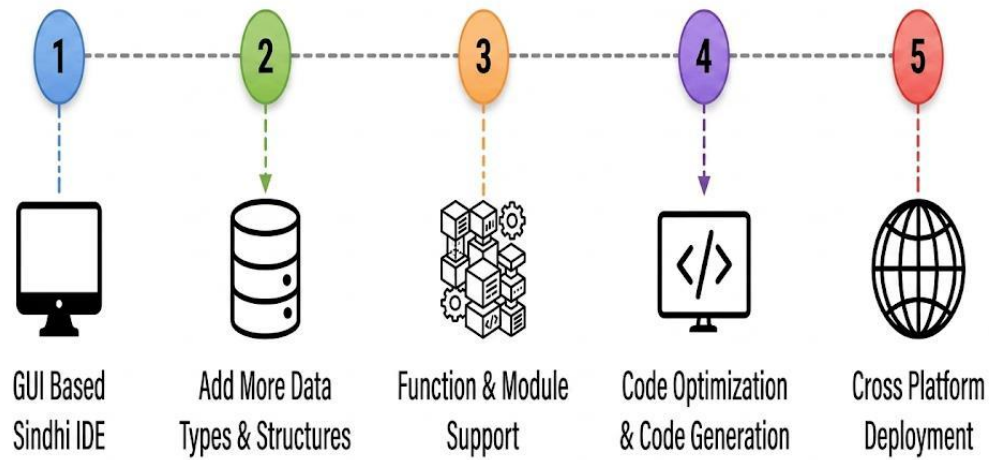


Figure 5.1: Future Enhancement Roadmap

5.5 FUTURE WORK

Future Area	Planned Enhancement
Advanced Data Types	Floating-point, Boolean, arrays, matrices and user-defined structures
Functions and Procedures	Parameters, local variables, return statements and recursion
Parser and Semantic Analysis	Parse trees/ASTs, type checking and scope validation
Integrated Development Environment	Syntax highlighting, completion, debugging and diagnostics
Compiler Optimization	Intermediate code, constant folding and dead-code elimination
Educational Integration	Use in schools, colleges, universities and online learning
Multilingual Support	Extension to additional regional languages

Table 5.1: Future Enhancement Areas

Although the proposed compiler successfully implements the primary objectives of the research, several improvements can be incorporated in future versions to address its current limitations and increase its functionality and educational value.

5.5.1 Support for Advanced Data Types

The current implementation primarily supports numeric and string variables. Future versions may include additional data types such as floating-point numbers, Boolean values, arrays, matrices, and user-defined structures to address this limitation. These additions would allow programmers to solve more complex computational problems while expanding the capabilities of the language.

5.5.2 Functions and Procedures

The present implementation executes programs sequentially without supporting reusable functions. Future research may incorporate user-defined functions, parameter passing, local variables, return statements, recursive function calls, and modular programming techniques. These features would significantly improve software organization and program reusability.

5.5.3 Parser and Semantic Analysis

The current system primarily focuses on lexical analysis and interpreter execution. Future versions may introduce a complete syntax parser capable of generating parse trees or Abstract Syntax Trees (ASTs). Semantic analysis could then verify variable declarations, type compatibility, scope management, and semantic correctness before program execution. Such enhancements would make the compiler more robust and closer to modern compiler implementations.

5.5.4 Integrated Development Environment (IDE)

The usability of the programming language can be improved through the development of a dedicated Integrated Development Environment (IDE). Such an environment may include syntax highlighting, automatic indentation, intelligent code completion, real-time syntax checking, debugging tools, project management, and compiler diagnostics. These features would significantly improve the programming experience for beginners.

5.5.5 Compiler Optimization

Future work may also focus on compiler optimization techniques. Intermediate code generation, dead-code elimination, constant folding, expression optimization, and improved memory management would increase execution efficiency while providing students with practical examples of advanced compiler construction concepts.

5.5.6 Educational Integration

The developed programming language has strong potential for integration into educational institutions. Universities, colleges, and schools may adopt the language as an introductory programming environment for Sindhi-speaking students. Online

learning platforms and e-learning systems could also incorporate the compiler to provide interactive programming education in the Sindhi language.

5.5.7 Multilingual Programming Support

The architecture of the proposed compiler can be extended to support additional regional languages such as Urdu, Punjabi, Balochi, and Pashto. By replacing the language-specific keywords while preserving the compiler architecture, the system could evolve into a multilingual educational programming platform that serves learners from different linguistic backgrounds.

5.6 FINAL REMARKS

The Latif Ji Boli Sindhi Programming Language Interpreter demonstrates that regional-language programming is both technically feasible and educationally valuable. By combining compiler construction principles with the Sindhi language, the project provides an innovative programming environment that reduces linguistic barriers while preserving the fundamental concepts of computer programming.

The successful implementation of lexical analysis, Unicode processing, token management, interpreter execution, and runtime error handling confirms that compiler technologies can effectively support indigenous languages. The project not only contributes to programming education but also promotes the digital preservation and technological use of the Sindhi language.

It is expected that this research will encourage future studies in multilingual compiler construction, regional-language programming environments, educational software development, and compiler optimization techniques. With continued research and development, the proposed system can evolve into a complete programming language capable of supporting advanced software development while maintaining its primary objective of making programming more accessible to Sindhi-speaking learners.

REFERENCES

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley, 2006.
- [2] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 3rd ed. Cambridge, MA, USA: Morgan Kaufmann, 2022.
- [3] A. V. Aho and J. D. Ullman, *Principles of Compiler Design*. Reading, MA, USA: Addison-Wesley, 1977.
- [4] The Unicode Consortium, *The Unicode Standard, Version 17.0.0*. South San Francisco, CA, USA: Unicode Consortium, 2025.
- [5] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004, doi: 10.2307/25148625.
- [6] K. Peffers, T. Tuunanen, C. Gengler, M. Rossi, W. Hui, V. Virtanen, and S. Bragge, “Design science research process: A model for producing and presenting information systems research,” in *Proc. Design Science Research in Information Systems and Technology*, 2007.
- [7] M. Resnick, K. D. Maloney, M. Monroy-Hernández, N. Rusk, E. Eastmond, K. Brennan, M. Millner, E. Rosenbaum, J. Silver, B. Silverman, and Y. Kafai, “Scratch: Programming for all,” *Communications of the ACM*, vol. 52, no. 11, pp. 60–67, 2009, doi: 10.1145/1592761.1592779.
- [8] R. W. Sebesta, *Concepts of Programming Languages*, 12th ed. Boston, MA, USA: Pearson, 2019.
- [9] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [10] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2019.
- [11] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. Thousand Oaks, CA, USA: SAGE Publications, 2018.
- [12] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman, *The Java Language Specification, Java SE 26 Edition*. Oracle, 2026.

- [13] Oracle, “Java Platform, Standard Edition 26 Documentation,” Oracle. Accessed: Aug. 17, 2026. [Online]. Available: <https://docs.oracle.com/en/java/javase/26/>
- [14] VMware, “Spring Boot Reference Documentation,” Spring Documentation. Accessed: Aug. 17, 2026. [Online]. Available: <https://docs.spring.io/spring-boot/>
- [15] Google, “Angular Documentation,” Angular. Accessed: Aug. 17, 2026. [Online]. Available: <https://angular.dev/>
- [16] B. W. Kernighan and D. M. Ritchie, The C Programming Language, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.
- [17] K. E. Wiegers and J. Beatty, Software Requirements, 3rd ed. Redmond, WA, USA: Microsoft Press, 2013.
- [18] P. J. Guo, “Non-Native English Speakers Learning Computer Programming: Barriers, Desires, and Design Opportunities,” in Proc. 2018 CHI Conference on Human Factors in Computing Systems, 2018, p. 396, doi: 10.1145/3173574.3173970.
- [19] C. Piech and S. Abu-El-Haija, “Human Languages in Source Code: Auto-Translation for Localized Instruction,” in Proc. ACM Conference on Learning at Scale, 2020, pp. 167–174, doi: 10.1145/3386527.3405916.
- [20] K. Reestman and B. Dorn, “Native Language’s Effect on Java Compiler Errors,” in Proc. 2019 ACM Conference on International Computing Education Research (ICER), 2019, pp. 249–257, doi: 10.1145/3291279.3339423.
- [21] Python Software Foundation, “The Python Language Reference,” Python Documentation. Accessed: Aug. 17, 2026. [Online]. Available: <https://docs.python.org/3/reference/>
- [22] Pakistan Bureau of Statistics, Provincial Census Report 2023: Sindh. Government of Pakistan, 2024.
- [23] P. Perera and S. Ahangama, “SimplyTrans: A Simplified Approach to Sinhala-Based Coding and Introductory Programming Language Localization,” in Proc. 2021 IEEE 16th International Conference on Industrial and Information Systems (ICIIS), 2021, pp. 318–323, doi: 10.1109/ICIIS53135.2021.9660709.